AI POWERED INTELLIGENT HOSPITAL PATIENT FLOW MANAGEMENT SYSTEM

A PROJECT REPORT

submitted in partial fulfilment of the requirements for the award of the degree of

[BACHELOR OF TECHNOLOGY]

Student	Hall Ticket No.
A. RAMU	23K91A6715
B. RUSHI	23K91A6720
B. RAJ KUMAR	23K91A6725
A. PRABHAS	23K91A6703

Under the guidance of

[GUIDE NAME]

[DESIGNATION]

Department of Computer Science and Engineering (Data Science)

[COLLEGE NAME]

[CITY]

[ACADEMIC YEAR]

CERTIFICATE

This is to certify that the project report entitled “AI Powered Intelligent Hospital Patient Flow Management System” is a bonafide record of the work carried out by A. RAMU, B. RUSHI, B. RAJ KUMAR, A. PRABHAS under the guidance of [GUIDE NAME], submitted to [COLLEGE NAME] in partial fulfilment of the requirements for the award of [DEGREE] during [ACADEMIC YEAR].

Project Guide: _____ Head of Department: _____

Principal: _____

DECLARATION

We declare that the work presented in this report, entitled “AI Powered Intelligent Hospital Patient Flow Management System,” is our original academic project work. Sources used for background, comparison and implementation have been acknowledged. The reported software measurements were obtained from this project’s artefacts and local execution; no clinical deployment or real patient study is claimed.

Signatures: A. RAMU _____ B. RUSHI _____ B. RAJ KUMAR _____ A. PRABHAS _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, [GUIDE NAME], [DESIGNATION], for guidance and constructive review. We thank [HOD NAME], Head of the Department, [PRINCIPAL NAME], Principal, and the faculty of [COLLEGE NAME] for the academic environment and facilities used for this work.

We also acknowledge the authors of the Alleviate base paper and the open-source communities behind Python, FastAPI, scikit-learn, SQLAlchemy, React and their supporting libraries.

ABSTRACT

Hospitals coordinate registration, queues, consultation, laboratory work, pharmacy, wards and intensive care through interdependent decisions. When these states are fragmented across calls, registers or isolated screens, staff cannot see one reliable operational picture. CareFlow is a full-stack academic prototype that issues digital tokens, tracks visits, manages laboratory and bed states, publishes live updates and provides role-specific dashboards.

Two machine-learning components assist the workflow. A Random Forest regressor estimates wait time from synthetic queue context, and a Gradient Boosting classifier estimates triage priority from synthetic vital-sign data labelled with a simplified early-warning rule. On deterministic test partitions the wait model achieved R-squared 0.951041, MAE 5.982 minutes and RMSE 8.580 minutes; triage achieved 96.80% accuracy and 0.898475 macro F1, but only 63% critical recall. The results therefore demonstrate software integration and reproducible synthetic evaluation, not clinical validity.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 15

1.1 Domain and Background 15

CHAPTER 1 INTRODUCTION 16

1.2 Motivation and Current Practice 16

CHAPTER 1 INTRODUCTION 17

1.3 Problem Statement and Need 17

CHAPTER 1 INTRODUCTION 18

1.4 Objectives and Contributions 18

CHAPTER 1 INTRODUCTION 19

1.5 Scope, Limitations and Organisation 19

CHAPTER 2 LITERATURE SURVEY 20

2.1 Overview 20

CHAPTER 2 LITERATURE SURVEY 21

2.2 Base Paper Analysis 21

CHAPTER 2 LITERATURE SURVEY 22

2.3 Related Work: Access and Recommendation 22

CHAPTER 2 LITERATURE SURVEY 23

2.4 Related Work: Personalisation and Conversational Support 23

CHAPTER 2 LITERATURE SURVEY 24

2.5 Waiting-Time Research and Comparison 24

CHAPTER 2 LITERATURE SURVEY 25

2.6 Research Gap 25

CHAPTER 3 FEASIBILITY STUDY 26

3.1 Technical and Economic Feasibility 26

CHAPTER 3 FEASIBILITY STUDY 27

3.2 Operational and Schedule Feasibility 27

CHAPTER 3 FEASIBILITY STUDY 28

3.3 Advantages and Constraints 28
CHAPTER 3 FEASIBILITY STUDY 29
3.4 Risk Assessment 29
CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS 30
4.1 Existing and Proposed Systems 30
CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS 31
4.2 Functional Requirements 31
CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS 32
4.3 Non-Functional Requirements 32
CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS 33
4.4 Hardware and Software Requirements 33
CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS 34
4.5 User Roles and Constraints 34
CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS 35
4.6 Requirement Traceability 35
CHAPTER 5 SYSTEM DESIGN 36
5.1 Overall System Architecture 36
CHAPTER 5 SYSTEM DESIGN 37
5.2 High-Level Workflow 37
CHAPTER 5 SYSTEM DESIGN 38
5.3 DFD Level 0 38
CHAPTER 5 SYSTEM DESIGN 39
5.4 DFD Level 1 39
CHAPTER 5 SYSTEM DESIGN 40
5.5 DFD Level 2 40
CHAPTER 5 SYSTEM DESIGN 41
5.6 Use-Case Design 41
CHAPTER 5 SYSTEM DESIGN 42
5.7 UML Class Design 42

CHAPTER 5 SYSTEM DESIGN 43
5.8 Sequence Design 43
CHAPTER 5 SYSTEM DESIGN 44
5.9 Database Design 44
CHAPTER 5 SYSTEM DESIGN 45
5.10 Database Schema 45
CHAPTER 5 SYSTEM DESIGN 46
5.11 Module Design 46
CHAPTER 5 SYSTEM DESIGN 47
5.12 User Interface Design 47
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 48
6.1 Overall Methodology 48
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 49
6.2 Dataset Source and Split 49
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 50
6.3 Preprocessing 50
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 51
6.4 Triage Algorithm 51
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 52
6.5 Wait-Time Algorithm 52
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 53
6.6 Training and Evaluation 53
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 54
6.7 Application Implementation 54
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 55
6.8 API, Security and Error Handling 55
CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION 56
6.9 Installation and Execution 56
CHAPTER 7 SYSTEM TESTING 57

7.1 Strategy and Limits....57
CHAPTER 7 SYSTEM TESTING....58
7.2 API Smoke Results....58
CHAPTER 7 SYSTEM TESTING....59
7.3 Frontend Static and Build Verification....59
CHAPTER 7 SYSTEM TESTING....60
7.4 Functional and Edge Observations....60
CHAPTER 7 SYSTEM TESTING....61
7.5 Security and Performance Boundaries....61
CHAPTER 7 SYSTEM TESTING....62
7.6 User Acceptance Limit....62
CHAPTER 8 RESULTS AND OUTPUT SCREENS....63
8.1 Public Waiting Board....63
CHAPTER 8 RESULTS AND OUTPUT SCREENS....64
8.2 Administrator Overview....64
CHAPTER 8 RESULTS AND OUTPUT SCREENS....65
8.3 Reception Output....65
CHAPTER 8 RESULTS AND OUTPUT SCREENS....66
8.4 Registration Input....66
CHAPTER 8 RESULTS AND OUTPUT SCREENS....67
8.5 Doctor Queue....67
CHAPTER 8 RESULTS AND OUTPUT SCREENS....68
8.6 Patient Detail....68
CHAPTER 8 RESULTS AND OUTPUT SCREENS....69
8.7 Laboratory Order....69
CHAPTER 8 RESULTS AND OUTPUT SCREENS....70
8.8 Admission Selection....70
CHAPTER 8 RESULTS AND OUTPUT SCREENS....71
8.9 Ward Capacity....71

CHAPTER 8 RESULTS AND OUTPUT SCREENS 72
8.10 ICU Capacity 72
CHAPTER 8 RESULTS AND OUTPUT SCREENS 73
8.11 Laboratory Workflow 73
CHAPTER 8 RESULTS AND OUTPUT SCREENS 74
8.12 Model Results 74
CHAPTER 8 RESULTS AND OUTPUT SCREENS 75
8.13 Confusion and Failure Analysis 75
CHAPTER 8 RESULTS AND OUTPUT SCREENS 76
8.14 API and Capacity Results 76
CHAPTER 8 RESULTS AND OUTPUT SCREENS 77
8.15 Objectives versus Outcomes 77
CHAPTER 9 CONCLUSION 78
9.1 Summary and Objectives Achieved 78
CHAPTER 9 CONCLUSION 79
9.2 Findings, Applications and Limitations 79
CHAPTER 10 FUTURE SCOPE 80
10.1 Improvements and Further Research 80
CHAPTER 10 FUTURE SCOPE 81
10.2 Scalability, Deployment and Integration 81
REFERENCES 82
REFERENCES 83
REFERENCES 84
APPENDIX A: IMPORTANT CODE 85
A.1 Queue ordering 85
APPENDIX A: IMPORTANT CODE 86
A.2 Wait prediction features 86
APPENDIX A: IMPORTANT CODE 87
A.3 Visit creation orchestration 87

APPENDIX A: IMPORTANT CODE 88

A.4 Role enforcement 88

APPENDIX B: INSTALLATION AND USER GUIDE 89

B.1 Local Installation 89

APPENDIX B: INSTALLATION AND USER GUIDE 90

B.2 Start and Sign In 90

APPENDIX B: INSTALLATION AND USER GUIDE 91

B.3 Role Workflows 91

APPENDIX C: ADDITIONAL SCREENS 92

C.1 Staff Sign-In 92

APPENDIX C: ADDITIONAL SCREENS 93

C.2 Registration Detail 93

APPENDIX C: ADDITIONAL SCREENS 94

C.3 Patient Detail 94

APPENDIX C: ADDITIONAL SCREENS 95

C.4 Admission Detail 95

LIST OF FIGURES

Figure 1. System architecture

Figure 2. High-level workflow

Figure 3. DFD Level 0

Figure 4. DFD Level 1

Figure 5. DFD Level 2

Figure 6. Use-case diagram

Figure 7. UML class diagram

Figure 8. Registration sequence

Figure 9. Entity-relationship diagram

Figure 10. Core algorithm flowchart

Figure 11. Triage confusion matrix

Figure 12. Measured model metrics

Figure 13. API latency

Figure 14. Public waiting board

Figure 15. Administrator dashboard

Figure 16. Reception dashboard

Figure 17. Doctor dashboard

Figure 18. Ward and ICU boards

Figure 19. Laboratory dashboard

LIST OF TABLES

Table 1. Literature comparison

Table 2. Feasibility summary

Table 3. Functional requirements

Table 4. Non-functional requirements

Table 5. User roles

Table 6. Database entities

Table 7. API groups

Table 8. Model results

Table 9. Smoke-test results

Table 10. Objectives versus outcomes

Table 11. Future scope

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
EHR	Electronic Health Record
ER	Entity Relationship
FHIR	Fast Healthcare Interoperability Resources
ICU	Intensive Care Unit
JWT	JSON Web Token
MAE	Mean Absolute Error
NEWS2	National Early Warning Score 2
OPD	Outpatient Department
REST	Representational State Transfer
RMSE	Root Mean Squared Error
UML	Unified Modeling Language

CHAPTER 1 INTRODUCTION

1.1 Domain and Background

Patient flow is the sequence of clinical and operational states through which a person moves after arriving at a hospital. Registration, queueing, consultation, tests, pharmacy, ward and intensive-care resources are coupled: a delay or stale state in one department propagates to others.

CareFlow treats that sequence as shared operational data. Its aim is not to diagnose disease but to make queues, resource state and inter-department transitions visible to the authorised people responsible for them.

CHAPTER 1 INTRODUCTION

1.2 Motivation and Current Practice

The supplied project brief identifies manual name calling, telephone bed checks and hand-managed laboratory queues as sources of uncertainty. Conventional hospital management software may digitise billing and registration without coordinating the internal journey in real time.

A useful prototype therefore needs one visit identity, a visible token, explicit status values, department ownership and live resource views. Prediction can support communication only when its evidence and limitations are shown honestly.

CHAPTER 1 INTRODUCTION

1.3 Problem Statement and Need

The problem is to coordinate a patient's in-hospital flow across OPD, laboratory, pharmacy, ward, ICU and emergency services while preserving priority and current state. Staff need to know who is waiting, what action is pending and whether a resource is available. Patients need a clear token and expected wait.

The need is operational: reduce information fragmentation, not to claim a measured clinical outcome. This report distinguishes implemented functionality, synthetic-model performance and unmeasured impact.

CHAPTER 1 INTRODUCTION

1.4 Objectives and Contributions

Table 1.1 Objectives.

Objective	Implemented evidence
Tokens	Department code plus daily sequence
Movement	MovementLog on registration/explicit transfer
Labs	Order and status routes
Beds	Availability, admit and release routes
AI assistance	Triage and wait predictors
Dashboards	Five roles plus public board

The objectives are to issue digital tokens, track visit movement, coordinate laboratory tasks, expose bed/ICU availability, assist priority and wait estimation, and provide role dashboards. The contributions are a unified Visit state, priority-aware queue ordering, two reproducible model artefacts, resource workflows, WebSocket refresh and a documented evidence boundary.

CHAPTER 1 INTRODUCTION

1.5 Scope, Limitations and Organisation

Scope ends at an academic local prototype. Billing, real EHR integration, diagnosis, production identity, notification providers, cloud scaling and clinical decision authority are excluded. Synthetic data, 63% critical recall, missing automated tests and production-security gaps are material limitations.

Chapters 2–4 review prior work, feasibility and requirements; Chapters 5–6 describe design and implementation; Chapters 7–8 report testing and results; Chapters 9–10 conclude and identify future work. References and three appendices follow.

CHAPTER 2 LITERATURE SURVEY

2.1 Overview

Related work spans patient-facing healthcare applications, disease or specialist recommendation, location services, personalised diet/exercise planning and waiting-time prediction. These streams show the value of integrated access but often stop before in-hospital resource coordination.

The survey uses the supplied Alleviate paper as the base paper and the sources already cited in the batch literature document. Reported figures remain attributed to their original tasks.

CHAPTER 2 LITERATURE SURVEY

2.2 Base Paper Analysis

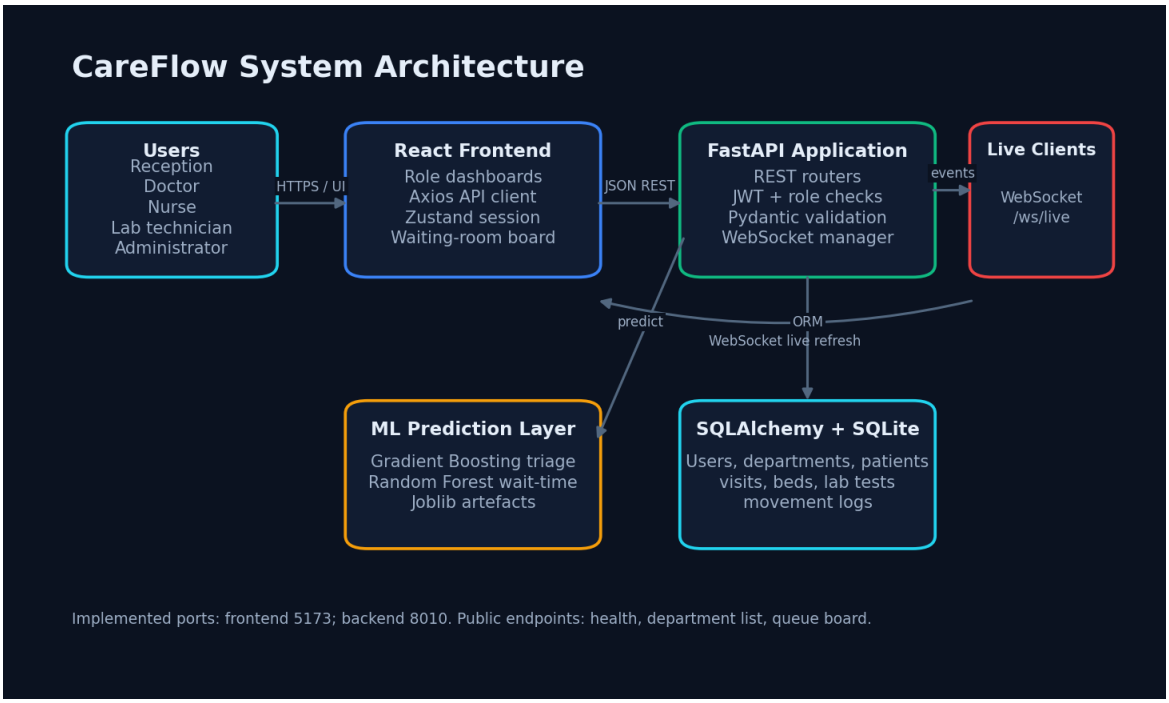


Figure 2.1 CareFlow architecture used for contrast with the base paper.

Alleviate combines Flutter/Dart, Flask and Firebase with specialist recommendation, prescription management, clinic dashboards, maps, appointments, diet/exercise planning and doctor verification. It reports 97.24% precision for its logistic-regression recommendation task and 84.76% precision for a diet-plan Random Forest.

Its focus is patient-side access and personalisation. CareFlow adopts the integrated dashboard principle but changes the problem to internal queues, visit state, laboratory work and bed capacity. The model tasks and datasets differ, so numerical comparison would be invalid.

CHAPTER 2 LITERATURE SURVEY

2.3 Related Work: Access and Recommendation

Tan et al. integrate symptom diagnosis, health records and hospital recommendation; Abinaya and Ganesan, Munir et al., and Nimbalkar and Fadnavis focus on locating appropriate or nearby facilities. Kumar, Gujar et al., and Bhanuteja et al. apply classification to disease or doctor recommendation.

These systems show that structured inputs and role-aware services can improve access. They do not by themselves represent the changing operational state of a patient already moving through a hospital.

CHAPTER 2 LITERATURE SURVEY

2.4 Related Work: Personalisation and Conversational Support

Mogaveera et al. and Tseng et al. combine monitoring with diet and exercise guidance. Tanwar et al., Surve et al., Mendapara et al. and Rahman et al. investigate chatbots, NLP and conversational symptom capture. Umbare et al. combine prediction and location features.

CareFlow deliberately excludes general diagnosis and chatbot advice. Its human-computer interaction is role-specific workflow: a small set of auditable actions backed by explicit database state.

CHAPTER 2 LITERATURE SURVEY

2.5 Waiting-Time Research and Comparison

Table 2.1 Comparison boundary.

Work/theme	Approach	Data	Reported result	Limitation for CareFlow
Alleviate	LR/RF integrated app	Paper-specific datasets	97.24% recommendation precision	Different task and dataset
Waiting-time studies	ML regression	Observed outpatient context	Source-specific	No shared local dataset
CareFlow	RF wait + GB triage	Synthetic generators	R2 0.951; accuracy 0.968	No clinical validation

The supplied survey identifies recent work on queue waiting-time prediction, outpatient waiting time and dissatisfaction-aware interpretable prediction. These studies motivate predictive communication but differ in institution, features and outcome definition.

CareFlow's regressor is trained on a designed queue generator rather than observed hospital arrivals. Its R-squared and error values demonstrate consistency with that generator only.

CHAPTER 2 LITERATURE SURVEY

2.6 Research Gap

The gap is a compact system that connects priority-aware tokens, department ownership, movement evidence, laboratory tasks, ward/ICU capacity and live staff views while making the evidence boundary explicit. CareFlow addresses that application gap.

The project does not claim novelty in NEWS2, Random Forests or Gradient Boosting. Its novelty is integrative and operational. Future research must replace synthetic generators with governed real data and evaluate impact prospectively.

CHAPTER 3 FEASIBILITY STUDY

3.1 Technical and Economic Feasibility

Table 3.1 Feasibility summary.

Dimension	Finding
Hardware	Intel i5-12500 and 15.7 GB RAM sufficient
Software	Open-source local stack
Data	Synthetic generators available; real data absent
Cost	Low prototype cost; production cost unmeasured

The prototype uses mature open-source libraries already available on a standard Windows workstation. FastAPI, SQLAlchemy, scikit-learn, React and SQLite reduce development cost. The saved model artefacts and local database fit comfortably within 15.7 GB RAM.

No paid cloud, messaging or data licence was required. This supports academic feasibility but does not estimate production hosting, support, compliance, integration or clinical-governance cost.

CHAPTER 3 FEASIBILITY STUDY

3.2 Operational and Schedule Feasibility

Each role sees a focused dashboard consistent with its work: registration, consultation, laboratory, bed management or oversight. Seeded demonstration records make the workflow visible without real patient data.

Operational adoption would still require workflow mapping, staff training, downtime procedures, ownership of state updates and governance. The checked-in repository contains no schedule baseline; therefore schedule feasibility is qualitative rather than a measured variance claim.

CHAPTER 3 FEASIBILITY STUDY

3.3 Advantages and Constraints

Advantages include one visit state, explicit queue ordering, traceable movement, visible capacity, reusable APIs and live refresh. Constraints include single-node persistence, development credentials, synthetic evidence, incomplete transition coupling and no production integration.

The system remains feasible as an academic proof of implementation. Its constraints are severe enough that deployment without a redesign would be irresponsible.

CHAPTER 3 FEASIBILITY STUDY

3.4 Risk Assessment

Primary risks are clinical misclassification, stale state, unauthorised access, privacy exposure, service interruption and misleading interpretation of synthetic metrics. Mitigations include clinician override, validation warnings, authenticated channels, audit trails, monitoring, backups and explicit evidence labels.

The current build implements only part of these mitigations. Risk reduction is a future engineering requirement, not a completed outcome.

CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS

4.1 Existing and Proposed Systems

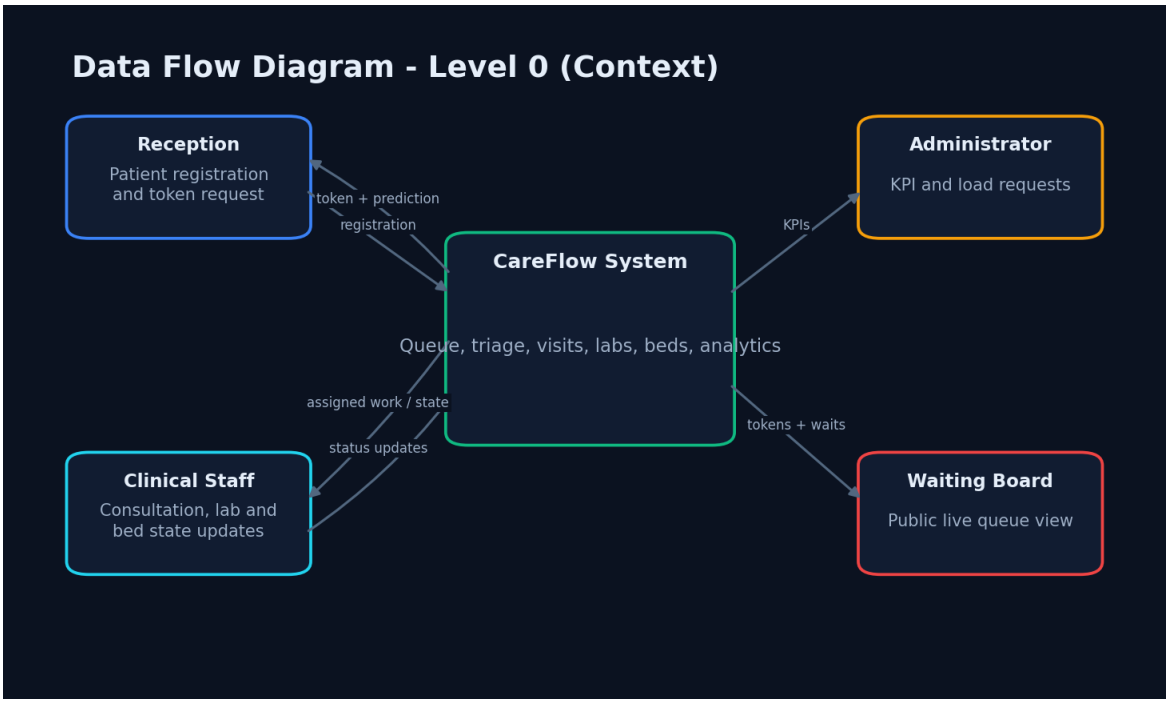


Figure 4.1 System context.

The existing condition described in the project brief uses manual or fragmented coordination. CareFlow proposes a shared visit record, digital token, prediction assistance, laboratory and bed modules, role dashboards and public queue display.

The proposed advantages are visibility and consistency. They were demonstrated through populated screens and API smoke tests; no real hospital comparison was conducted.

CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS

4.2 Functional Requirements

Table 4.1 Functional requirements.

ID	Requirement	Implementation
FR1	Authenticate staff	JWT login and role checks
FR2	Register visit	POST /api/visits
FR3	Issue token	Department code and sequence
FR4	Prioritise queue	Classifier plus priority sort
FR5	Manage laboratory work	Lab endpoints and board
FR6	Manage bed state	Bed endpoints and nurse board
FR7	Track movement	MovementLog
FR8	Report live state	KPIs, queue board, WebSocket

CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS

4.3 Non-Functional Requirements

Table 4.2 Non-functional requirements.

Category	Requirement	Current evidence / gap
Performance	Responsive local reads	Mean 3.985–14.231 ms; no load test
Security	Authenticated staff writes	JWT and role checks; WebSocket public
Reliability	Consistent persisted state	SQLite transaction; no failover
Usability	Role-focused screens	Real captures; no UAT
Maintainability	Modular code	Routes/services/models separated
Portability	Documented local run	Windows verified only

CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS

4.4 Hardware and Software Requirements

Measured hardware was a 12th Gen Intel Core i5-12500 with 6 cores, 12 logical processors and 15.7 GB RAM. The development environment used Windows 11 Pro, Python 3.11.9, Node/Vite, Google Chrome and the local filesystem.

Backend packages include FastAPI 0.115.0, SQLAlchemy 2.0.35 and scikit-learn 1.5.2. Frontend packages include React 19, TypeScript, Vite, Axios, Zustand, Recharts and Tailwind CSS.

CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS

4.5 User Roles and Constraints

Table 4.3 User roles.

Role	Principal permissions
Reception	Register and view visits
Doctor	Queue, consultation, lab order, admission, discharge
Nurse	Visit state and bed management
Lab technician	Progress laboratory tests
Administrator	Hospital-wide read/write access
Public viewer	Queue board only

CHAPTER 4 SYSTEM REQUIREMENTS AND ANALYSIS

4.6 Requirement Traceability

Every implemented objective maps to routes, database entities and one or more output screens. The movement objective is partial because not every implicit transition creates a movement record. Laboratory scheduling is a status workflow, not an optimisation algorithm. Waiting-time reduction is not measured.

This traceability prevents a feature shown in marketing copy from being reported as an achieved research result.

CHAPTER 5 SYSTEM DESIGN

5.1 Overall System Architecture

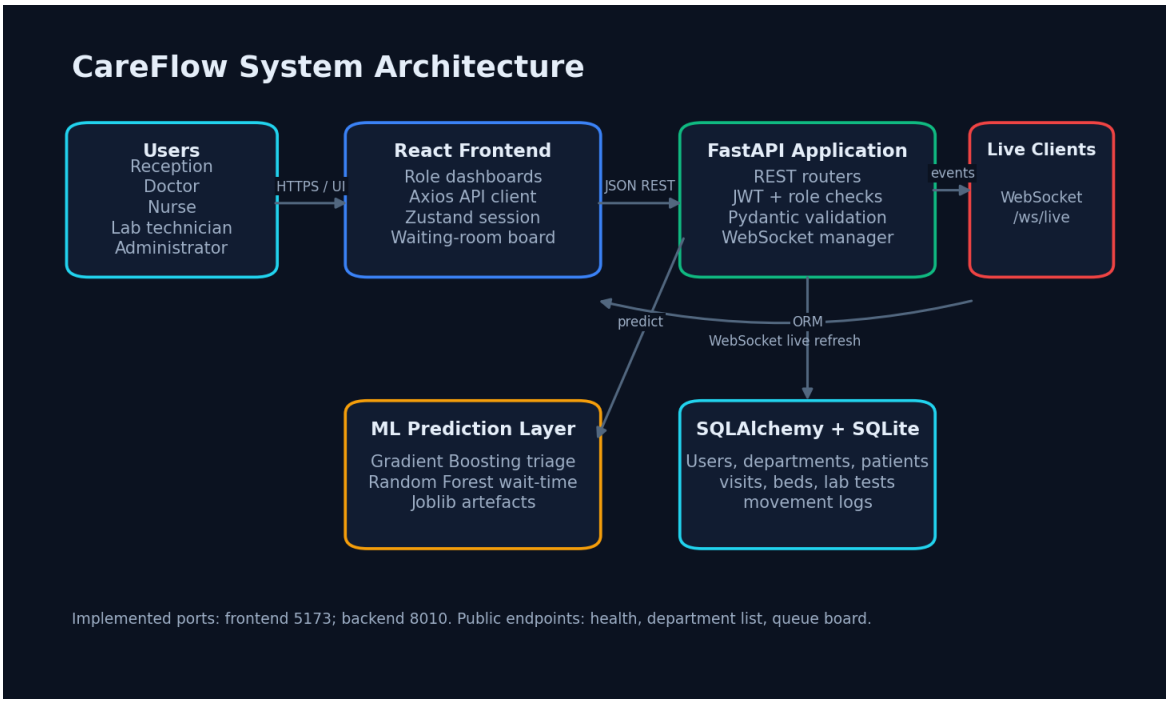


Figure 5.1 System architecture

The client, API, prediction layer, persistence and live channel form five replaceable concerns.

CHAPTER 5 SYSTEM DESIGN

5.2 High-Level Workflow

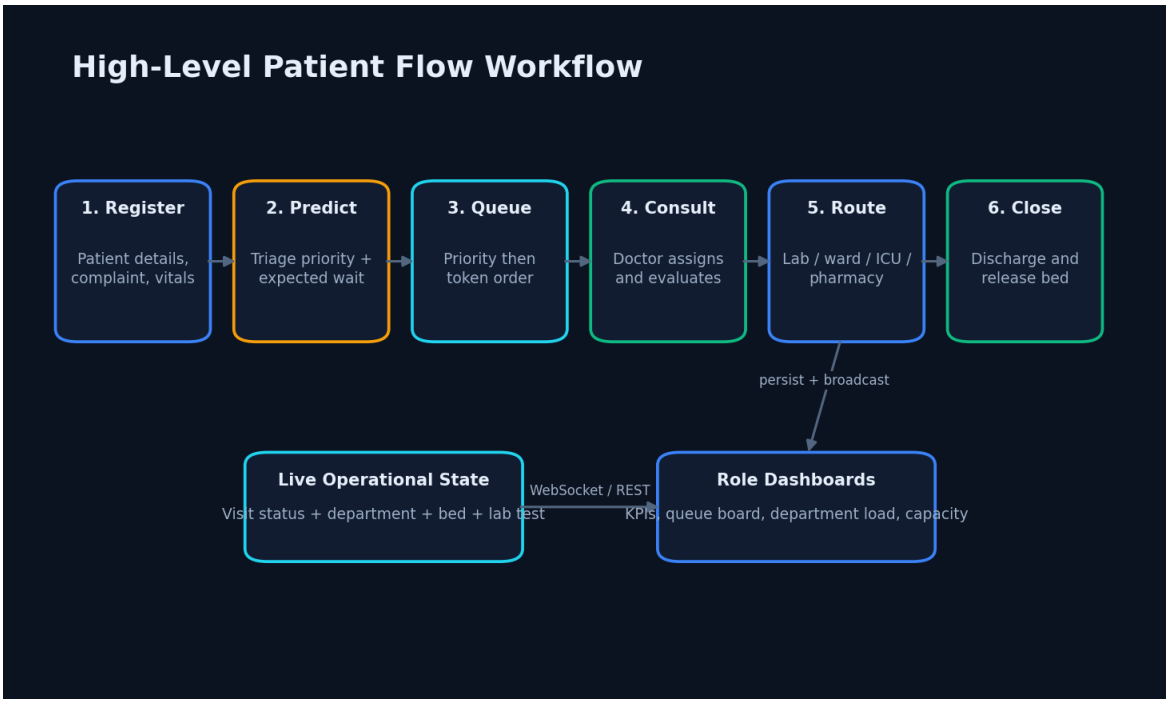


Figure 5.2 High-level workflow

Registration creates a predicted and prioritised visit that staff route to completion.

CHAPTER 5 SYSTEM DESIGN

5.3 DFD Level 0

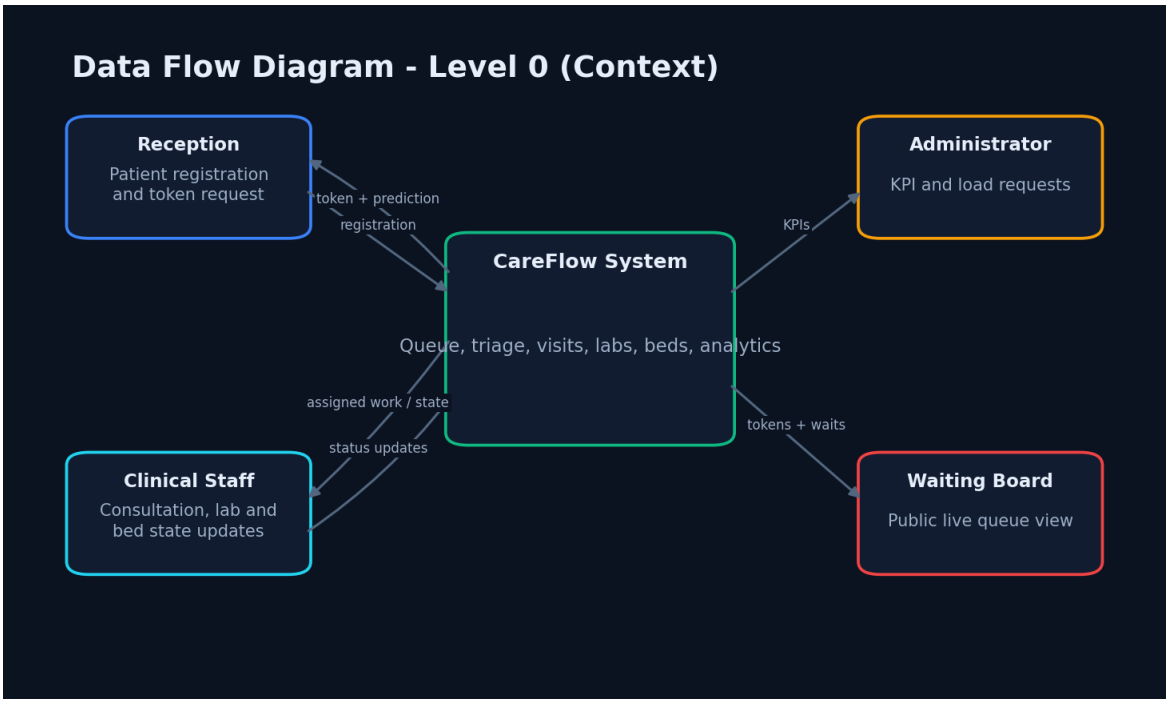


Figure 5.3 DFD Level 0

External actors exchange bounded operational data with the system.

CHAPTER 5 SYSTEM DESIGN

5.4 DFD Level 1

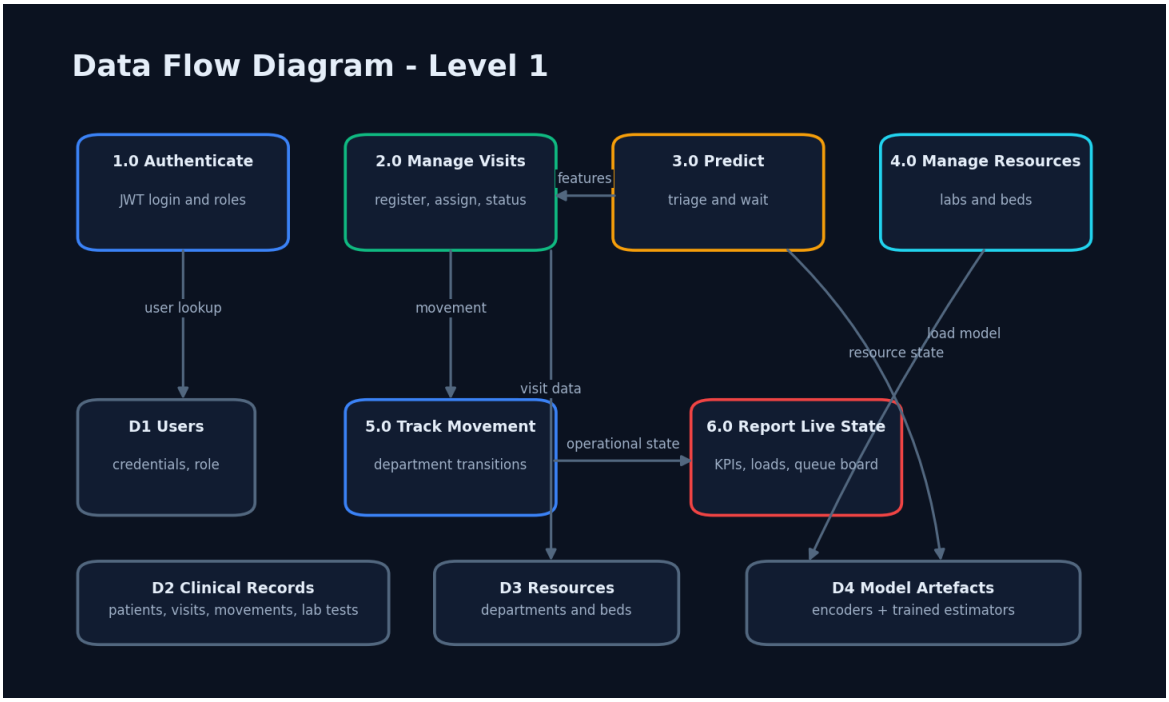


Figure 5.4 DFD Level 1

Authentication, visit, prediction, resource, movement and reporting processes use four data stores.

CHAPTER 5 SYSTEM DESIGN

5.5 DFD Level 2

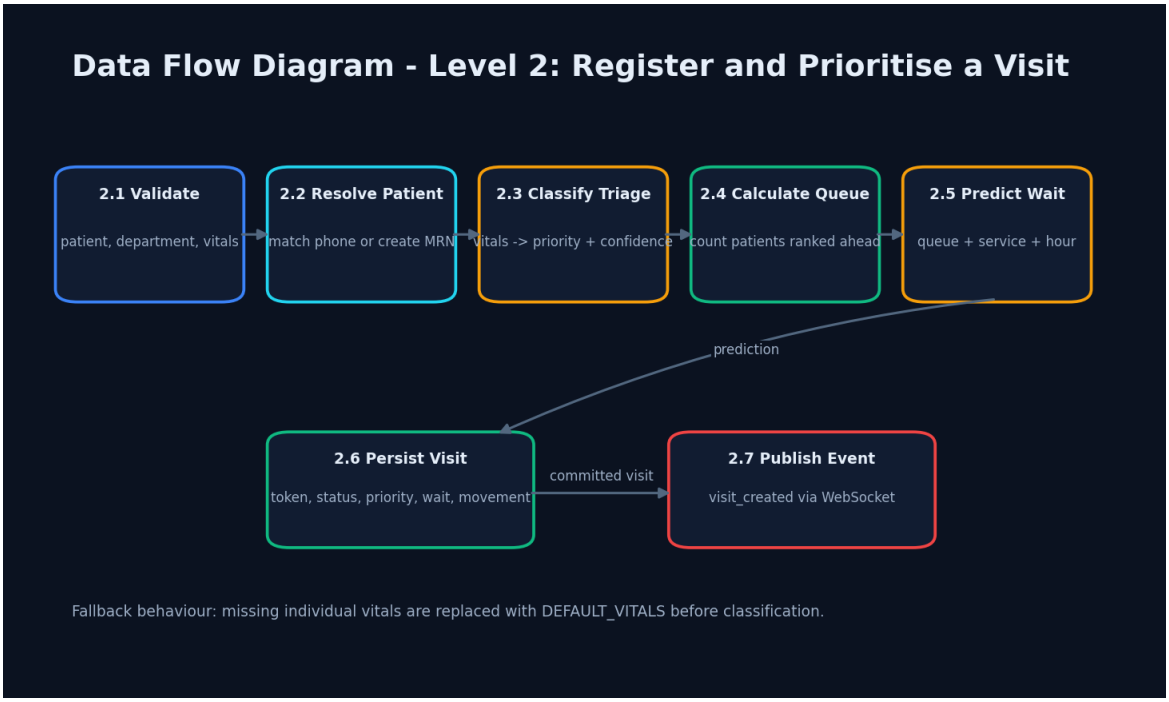


Figure 5.5 DFD Level 2

The visit-creation process is decomposed into validation, identity, inference, queue calculation, persistence and publishing.

CHAPTER 5 SYSTEM DESIGN

5.6 Use-Case Design

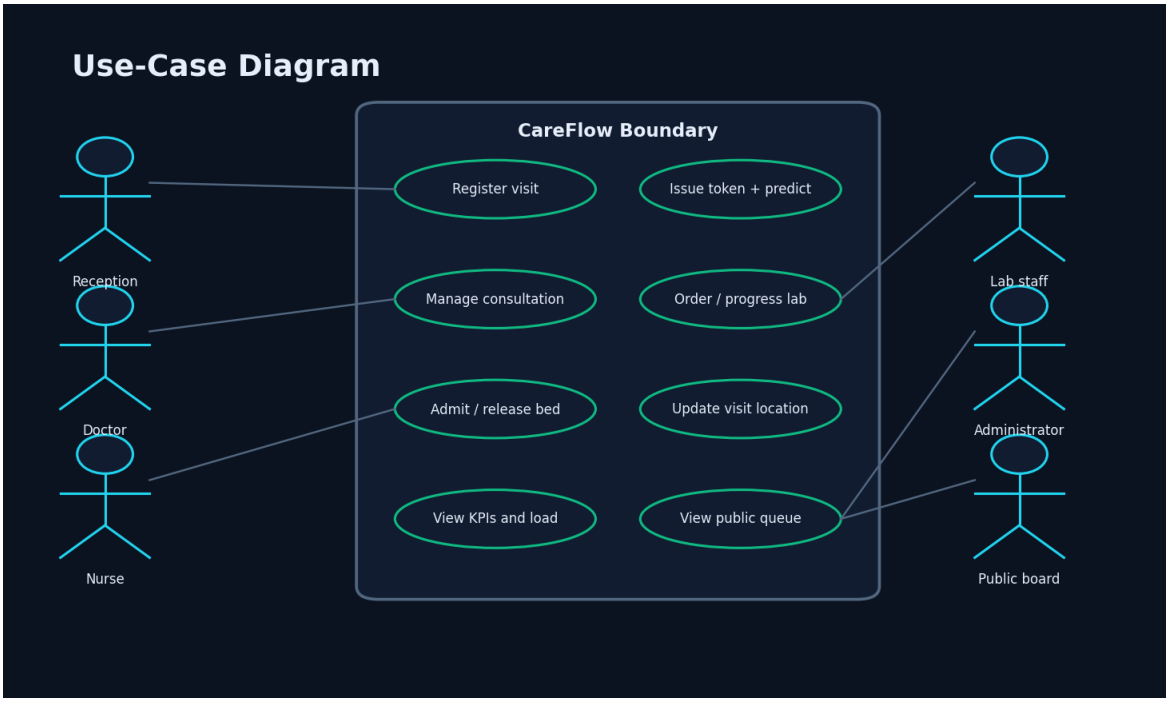


Figure 5.6 Use-case diagram

Use cases reflect the implemented role permissions and public board.

CHAPTER 5 SYSTEM DESIGN

5.7 UML Class Design

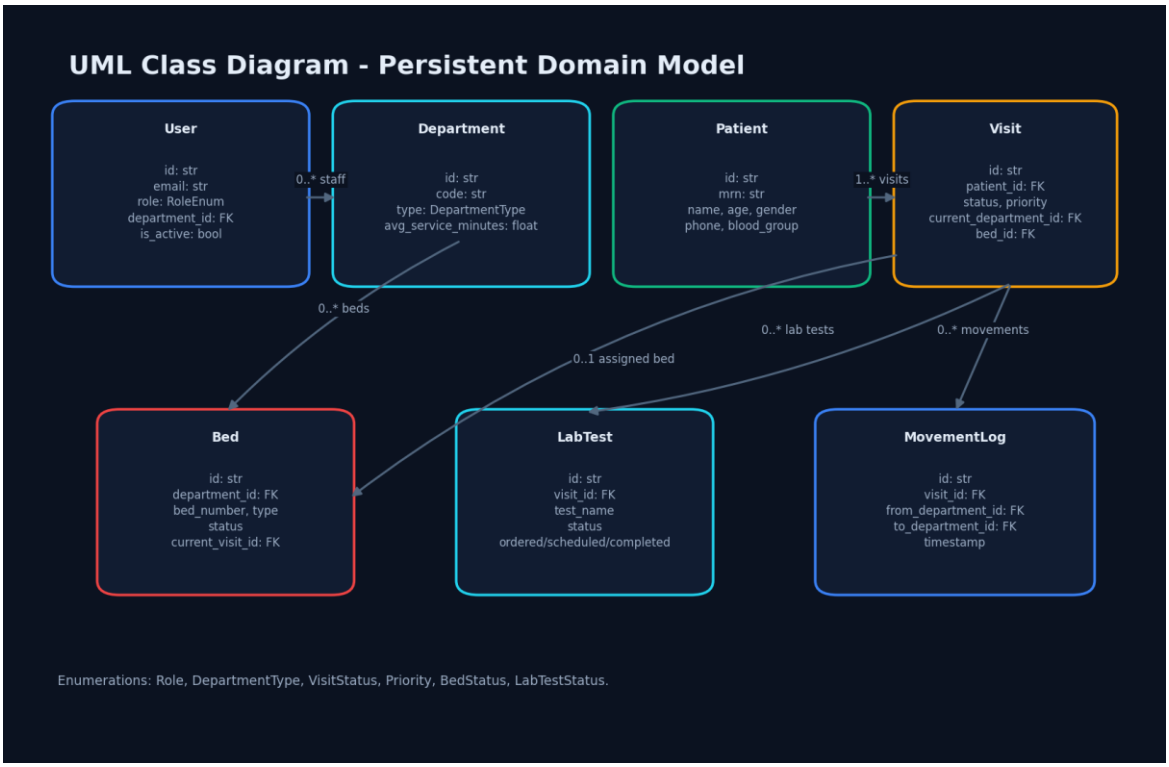


Figure 5.7 UML class diagram

Persistent domain classes and their relationships are shown without invented services.

CHAPTER 5 SYSTEM DESIGN

5.8 Sequence Design

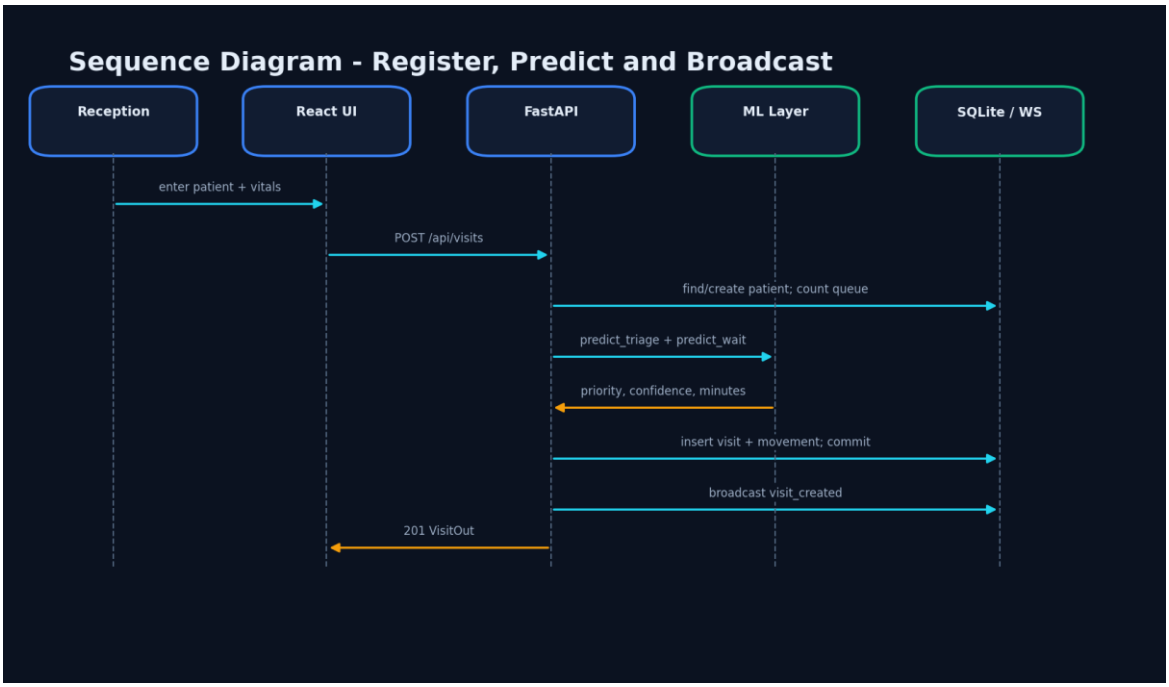


Figure 5.8 Registration sequence

A visit request crosses the UI, API, model, database and live-update channel.

CHAPTER 5 SYSTEM DESIGN

5.9 Database Design

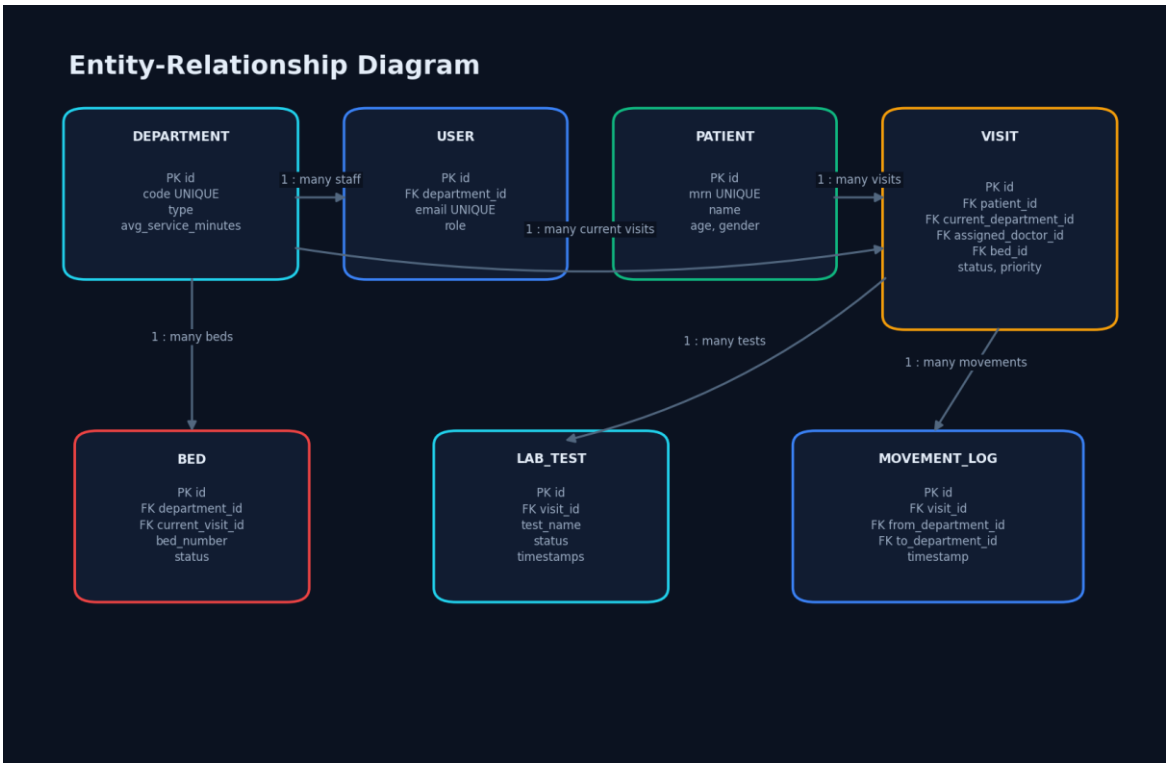


Figure 5.9 Entity-relationship diagram

Foreign keys connect the seven persistent entities.

CHAPTER 5 SYSTEM DESIGN

5.10 Database Schema

Table 5.1 Schema relationships.

Entity	Key relationships
User	department_id -> Department
Visit	patient, department, doctor, bed
Bed	department and optional current visit
LabTest	visit_id -> Visit
MovementLog	visit and from/to departments

Identifiers are 12-character hexadecimal strings. User email, department code and patient MRN are unique. Visits reference patients, departments, optional doctors and beds. Tests and movements are visit children. JSON stores vitals.

CHAPTER 5 SYSTEM DESIGN

5.11 Module Design

Backend modules separate configuration, database, security, models, schemas, route groups, queue rules, WebSocket handling and ML prediction. Frontend modules separate layout, common controls, API functions, session/toast stores, live-socket handling and role pages.

The central coupling remains Visit, which intentionally carries the state shared across modules.

CHAPTER 5 SYSTEM DESIGN

5.12 User Interface Design

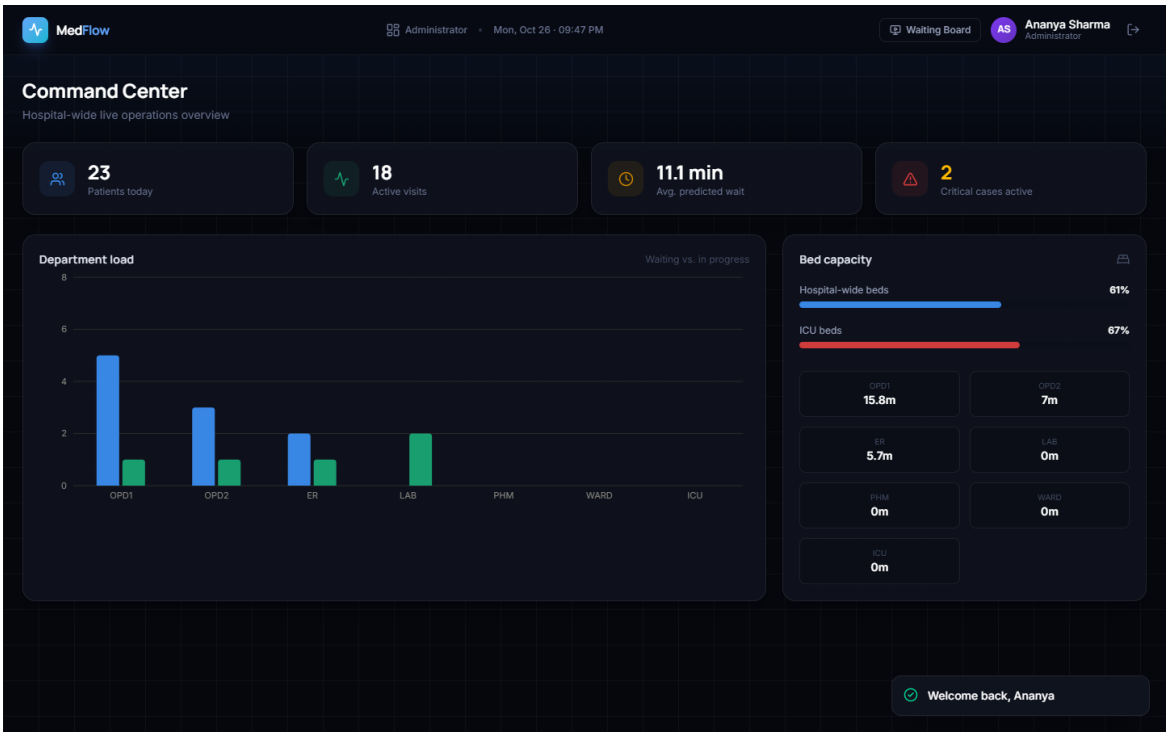


Figure 5.10 Administrator interface example.

The interface uses a dark operational palette, cards, status badges, compact tables and workflow modals. Admin screens favour aggregate visibility; clinical screens favour actionable lists; the public board uses large tokens for distance viewing.

The captured nurse screen also exposed a real initial-tab defect, demonstrating why visual verification is necessary in addition to code review.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.1 Overall Methodology

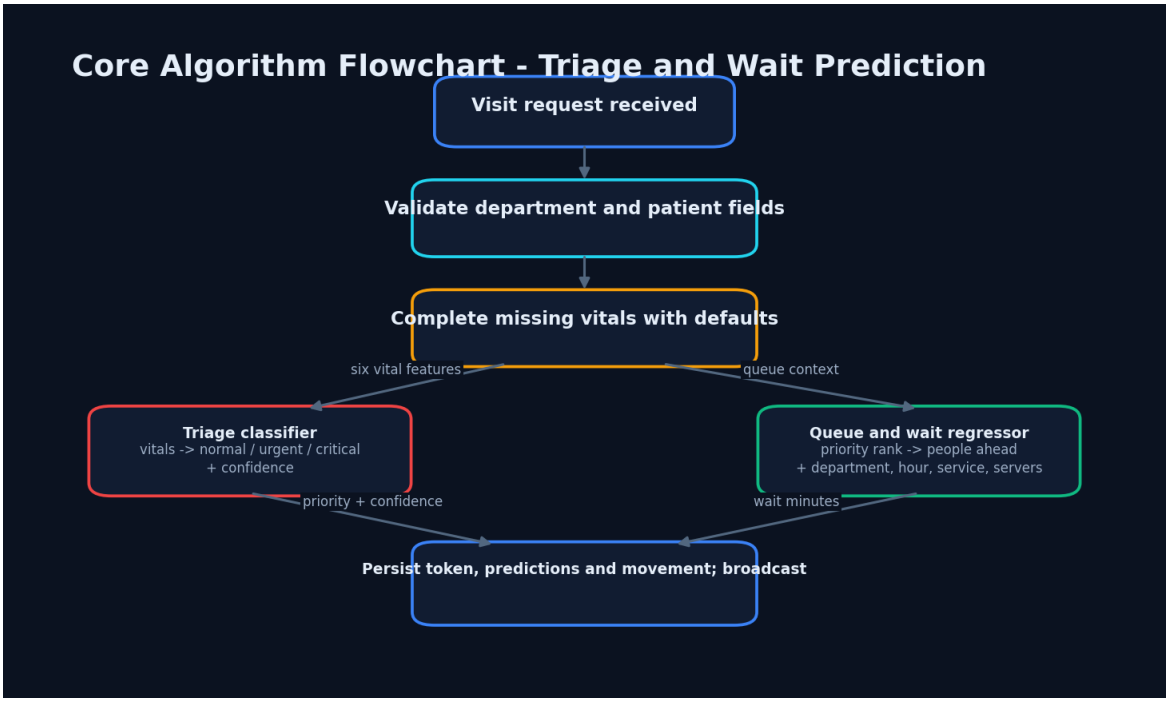


Figure 6.1 Core algorithm flow.

The method comprised fact extraction, code inspection, deterministic model evaluation, additive demonstration data, running-system capture and evidence-controlled documentation. The software pipeline performs validation, prediction, queue allocation, persistence and live publication.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.2 Dataset Source and Split

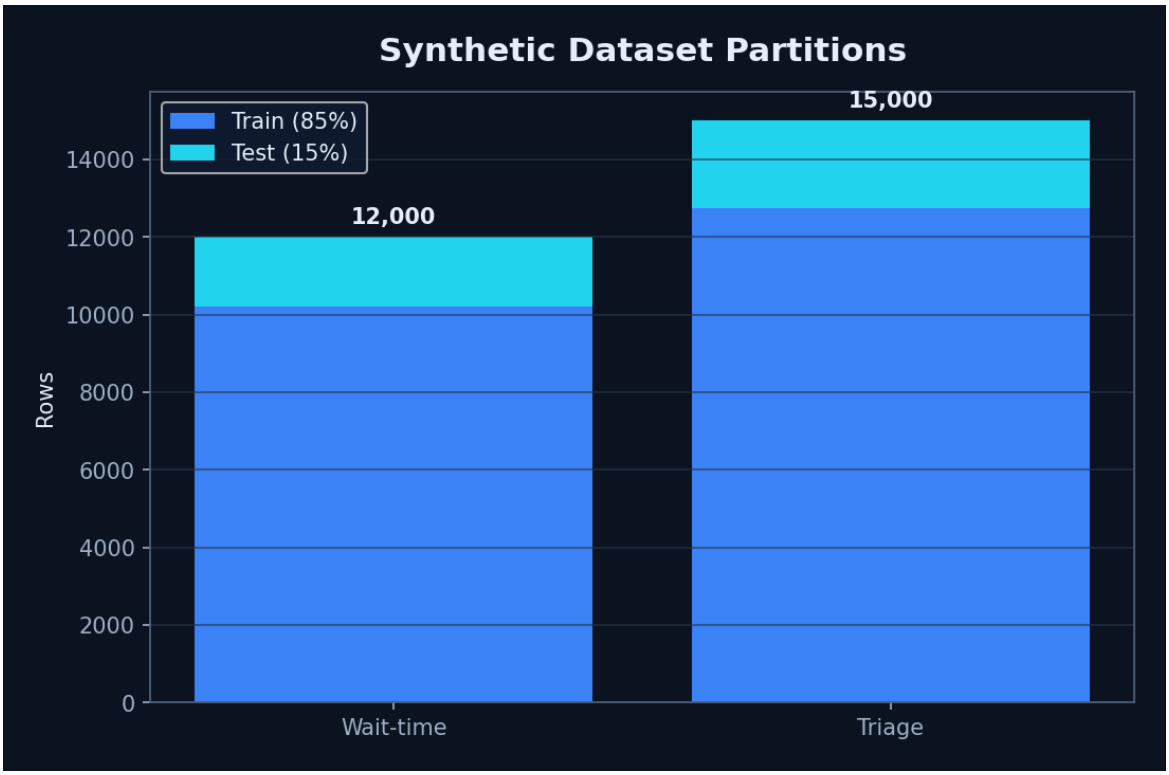


Figure 6.2 Dataset partitions.

Both datasets are generated in train.py with NumPy RNG seed 42. Wait data contains 12,000 rows and triage data 15,000 rows. Each is split 85/15, giving 10,200/1,800 wait rows and 12,750/2,250 triage rows.

No external licence applies to generated records and no real patient information is used.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.3 Preprocessing

Department type and priority are encoded with stored LabelEncoder instances. Tree models consume numeric values directly. The inference path preserves training feature names in pandas DataFrames. Missing vital components are filled from DEFAULT_VITALS: heart rate 78, systolic 118, diastolic 76, SpO2 97, temperature 36.9 and respiratory rate 16.

The silent defaulting is convenient for a demonstration but should become an explicit uncertainty state in a clinical redesign.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.4 Triage Algorithm

A simplified early-warning function assigns points for extreme respiratory rate, oxygen saturation, temperature, systolic pressure, heart rate and age. Total scores 0–3 are normal, 4–6 urgent and 7 or above critical. A 150-estimator Gradient Boosting classifier learns these labels.

The function is described as NEWS2-style rather than an exact clinical NEWS2 implementation. The saved classifier returns class probability as confidence.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.5 Wait-Time Algorithm

Synthetic target wait is based on queue ahead divided by servers, multiplied by average service duration, a 1.35 rush factor during defined hours, a priority factor and Gaussian noise. Results are clipped to 0–240 minutes. The Random Forest uses 200 trees and maximum depth 10.

Runtime inference defaults to two servers and current UTC hour. Department average service time is stored in the database.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.6 Training and Evaluation

The test partitions are deterministic and the existing Joblib models were measured without retraining. Wait R-squared was 0.951041, MAE 5.982 minutes and RMSE 8.580 minutes. Triage accuracy was 0.9680, macro precision 0.962435, macro recall 0.858574 and macro F1 0.898475.

These figures are reproducibility evidence within the generator, not clinical generalisation.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.7 Application Implementation

FastAPI creates database tables on import and verifies model artefacts at startup. Route functions use SQLAlchemy sessions through dependency injection. Pydantic models validate request ranges and response shape. React pages call a shared Axios client and refresh on selected WebSocket events.

The seed script is destructive by design, but documentation data was inserted additively because existing content had to be preserved.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.8 API, Security and Error Handling

JWT authentication, bcrypt passwords and route-level role checks provide the implemented security boundary. CORS accepts configured origins and a localhost regular expression for development. HTTP exceptions represent invalid credentials, permissions, missing records and unavailable beds.

Production controls—TLS termination, secret rotation, authenticated WebSockets, privacy logging, rate limits and least-privilege database accounts—are not implemented.

CHAPTER 6 METHODOLOGY AND SOFTWARE IMPLEMENTATION

6.9 Installation and Execution

Create or activate the backend virtual environment, install requirements, ensure data exist, then run `python -m uvicorn app.main:app --port 8010`. In frontend, run `npm install` and `npm run dev`; Vite serves port 5173 and proxies API and WebSocket requests to 8010.

The repository has no run.bat. Detailed user steps are in Appendix B.

CHAPTER 7 SYSTEM TESTING

7.1 Strategy and Limits

The repository contains no automated unit or integration tests. Verification therefore combined deterministic model evaluation, frontend lint/build, API smoke/latency requests, WebSocket connection and real-browser workflow capture.

This strategy verifies availability and presentation but cannot substitute for automated regression, concurrency, penetration, clinical or user-acceptance testing.

CHAPTER 7 SYSTEM TESTING

7.2 API Smoke Results

Table 7.1 Smoke results.

Evidence	Result
Login	HTTP 200
Read requests	150/150 HTTP 200
Mean endpoint range	3.985–14.231 ms
WebSocket connect	46.657 ms

Login returned HTTP 200. Ten read endpoints received 15 sequential requests each, for 150 successful HTTP 200 responses. The WebSocket connection also succeeded.

CHAPTER 7 SYSTEM TESTING

7.3 Frontend Static and Build Verification

Oxlint exited zero with 14 warnings concerning ref access, state updates in effects, missing dependencies and render-time randomness. TypeScript and Vite built successfully, transforming 3,573 modules in 1.73 seconds.

Two JavaScript chunks exceeded 500 kB after minification, indicating a code-splitting and performance opportunity.

CHAPTER 7 SYSTEM TESTING

7.4 Functional and Edge Observations

Authenticated dashboards loaded populated data; tokens, priorities, predicted waits, lab states and bed states were visible. Invalid or missing records have explicit HTTP errors in route code. Bed admission rejects non-available beds.

Observed edges include a blank initial nurse content panel until a tab click and incomplete coupling between laboratory test and visit status transitions.

CHAPTER 7 SYSTEM TESTING

7.5 Security and Performance Boundaries

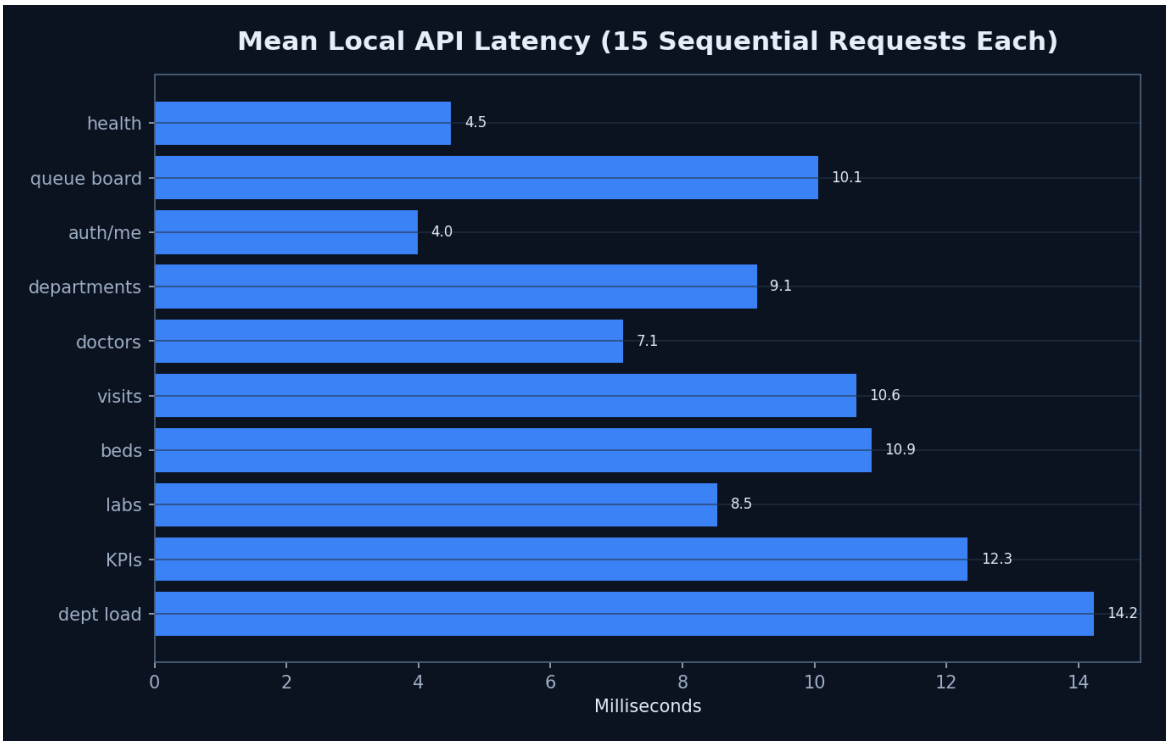


Figure 7.1 Local mean read latency.

JWT and role checks were exercised indirectly through successful role logins and protected data loads. No unauthorised access matrix, penetration test, dependency scan or privacy review was executed. The unauthenticated WebSocket is a known gap.

Latency values come from one sequential loopback client; no throughput, tail latency under load, memory saturation or multi-process behaviour was measured.

CHAPTER 7 SYSTEM TESTING

7.6 User Acceptance Limit

The screenshots show real application output with synthetic demonstration data, but no hospital staff or independent user group evaluated usefulness, clarity or workflow fit. Therefore user acceptance is unmeasured and is listed as future work.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.1 Public Waiting Board

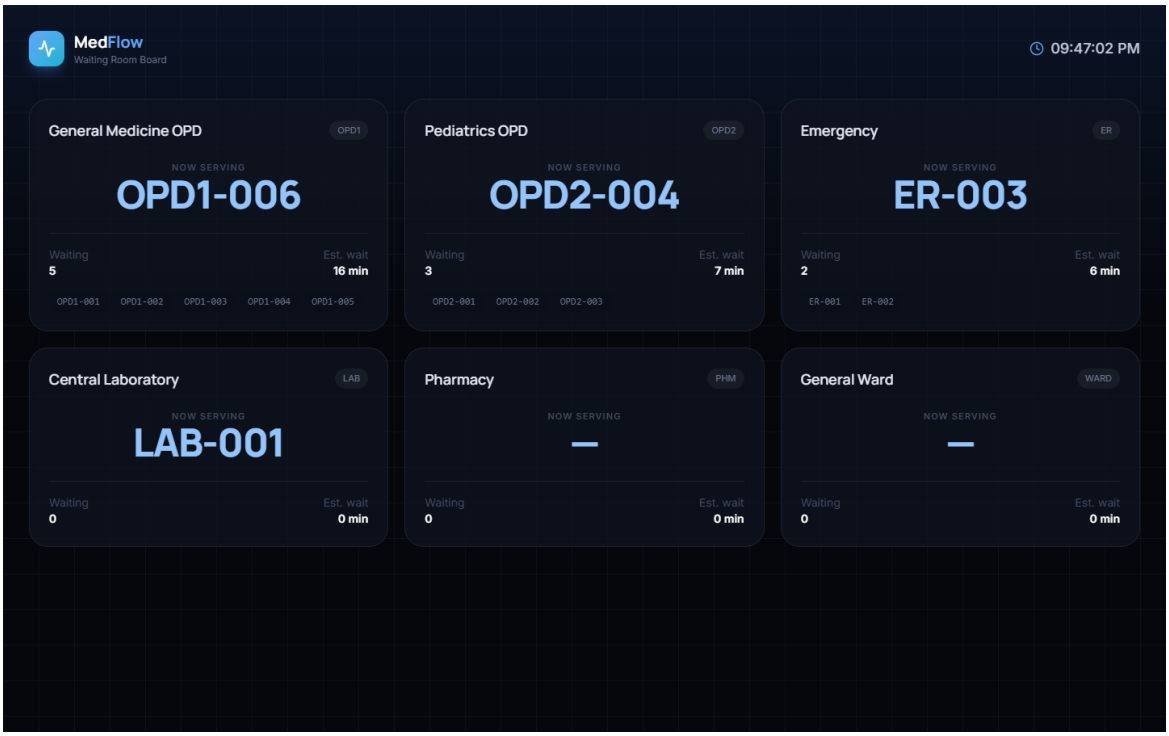


Figure 8.1 Public waiting board

Large token cards make current and upcoming queue state visible without authentication.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.2 Administrator Overview

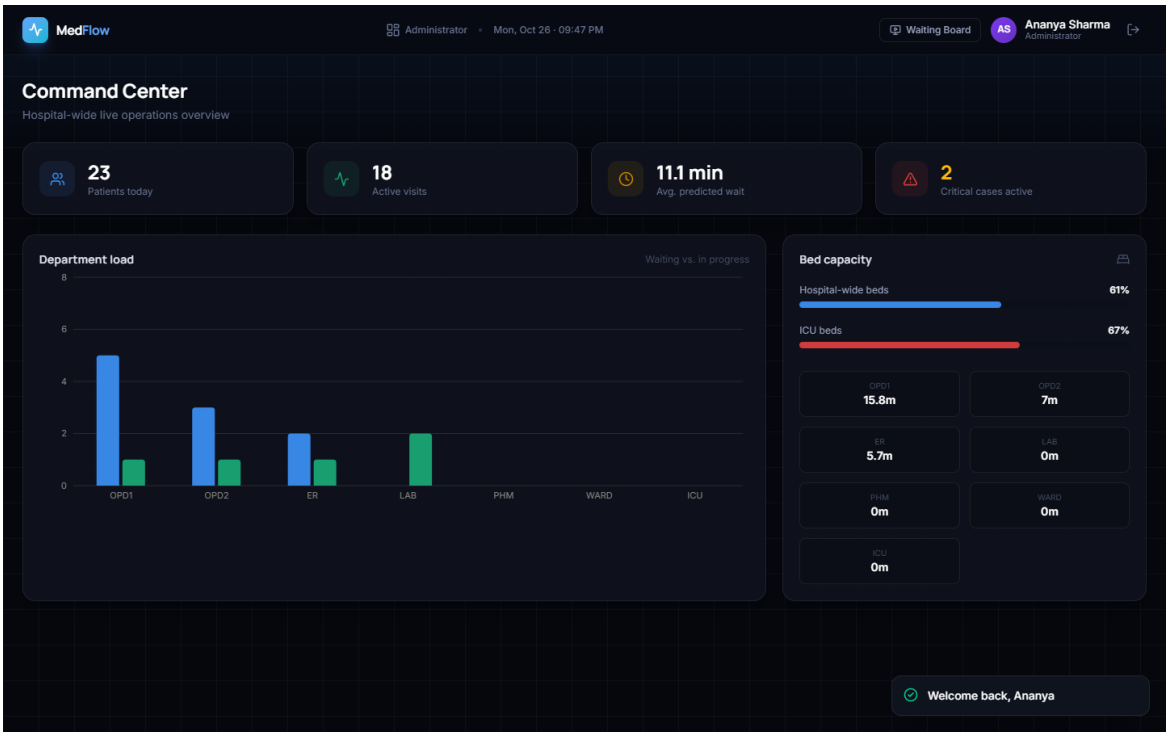


Figure 8.2 Administrator dashboard

The populated snapshot combines KPI, department and capacity evidence.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.3 Reception Output

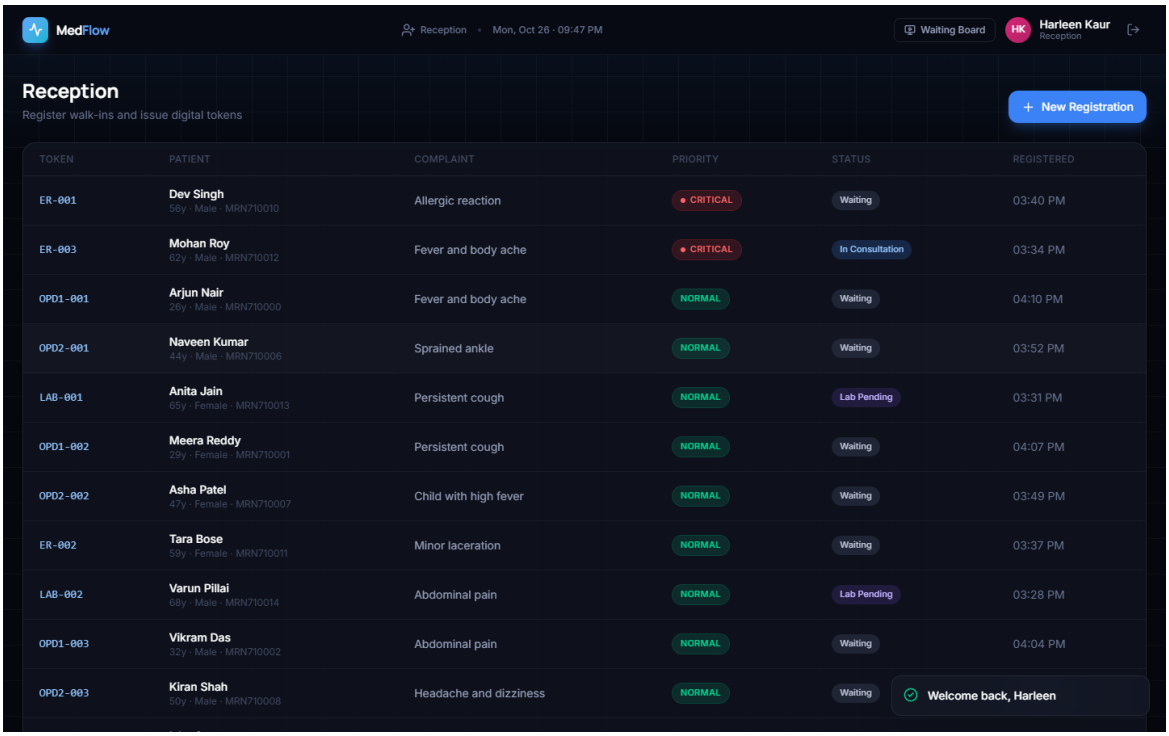


Figure 8.3 Reception dashboard

The registration list shows the operational identity of each visit.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.4 Registration Input

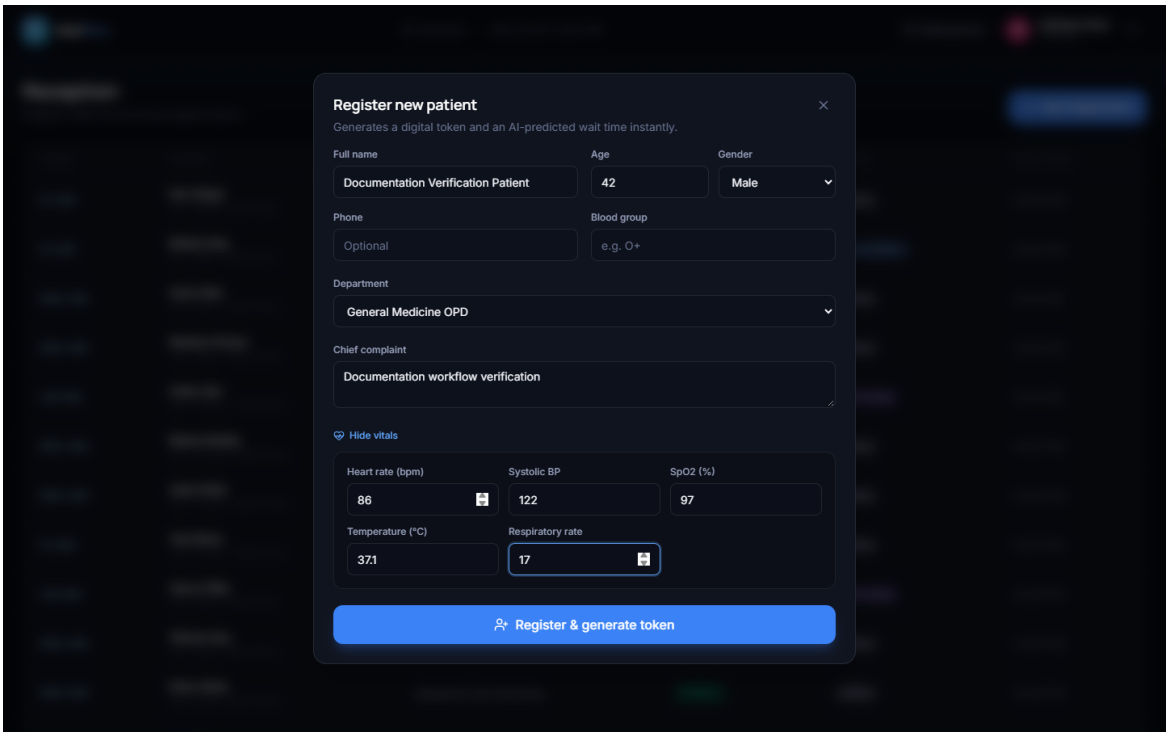


Figure 8.4 Registration form

Vitals are optional inputs to triage prediction.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.5 Doctor Queue

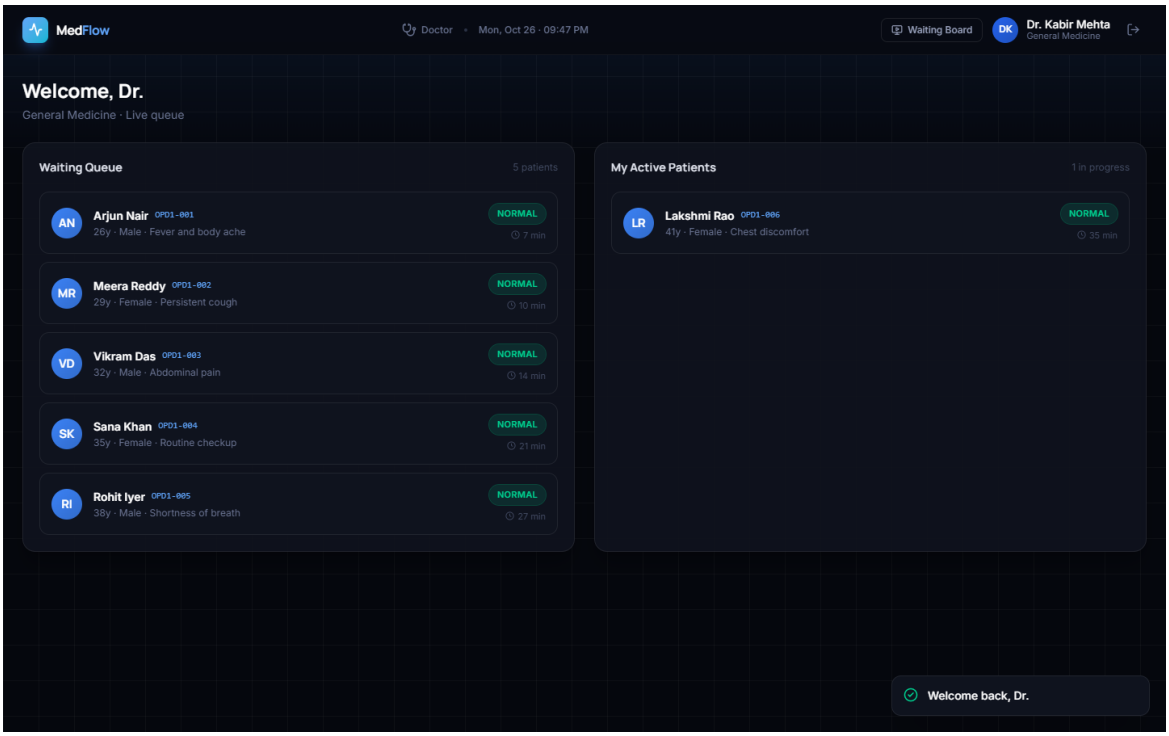


Figure 8.5 Doctor dashboard

Waiting and active visits are separated.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.6 Patient Detail

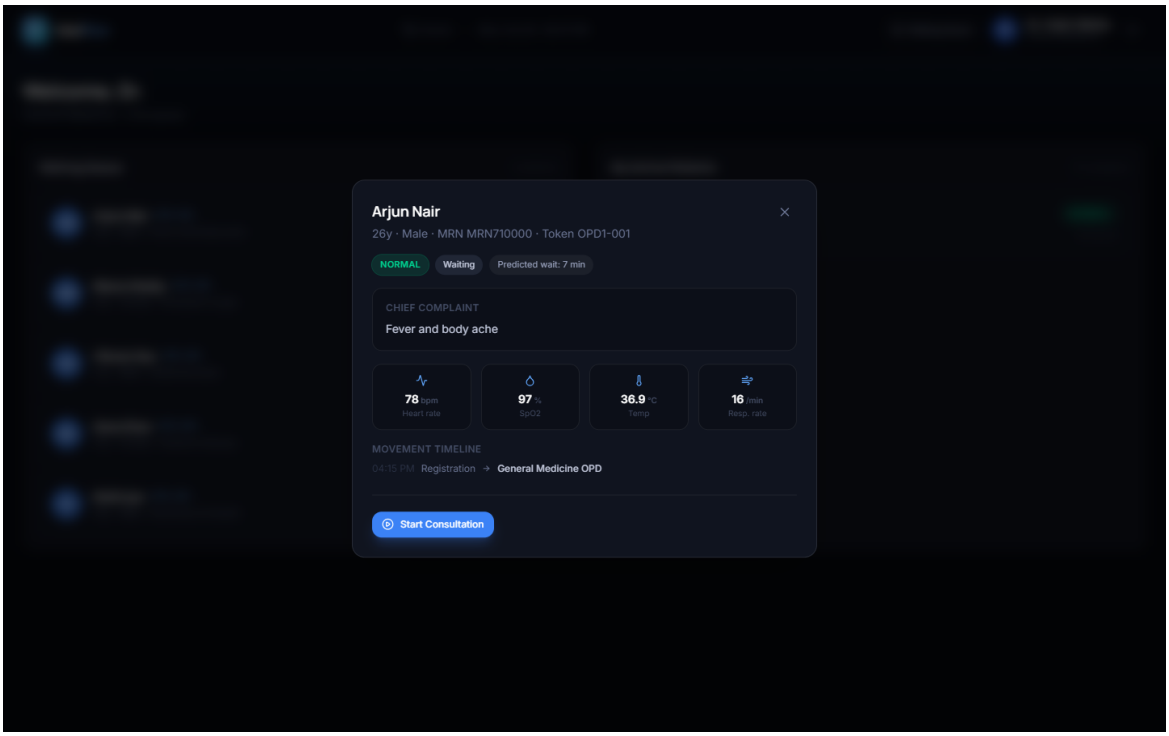


Figure 8.6 Patient detail

The modal shows complaint, vital context and movement.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.7 Laboratory Order

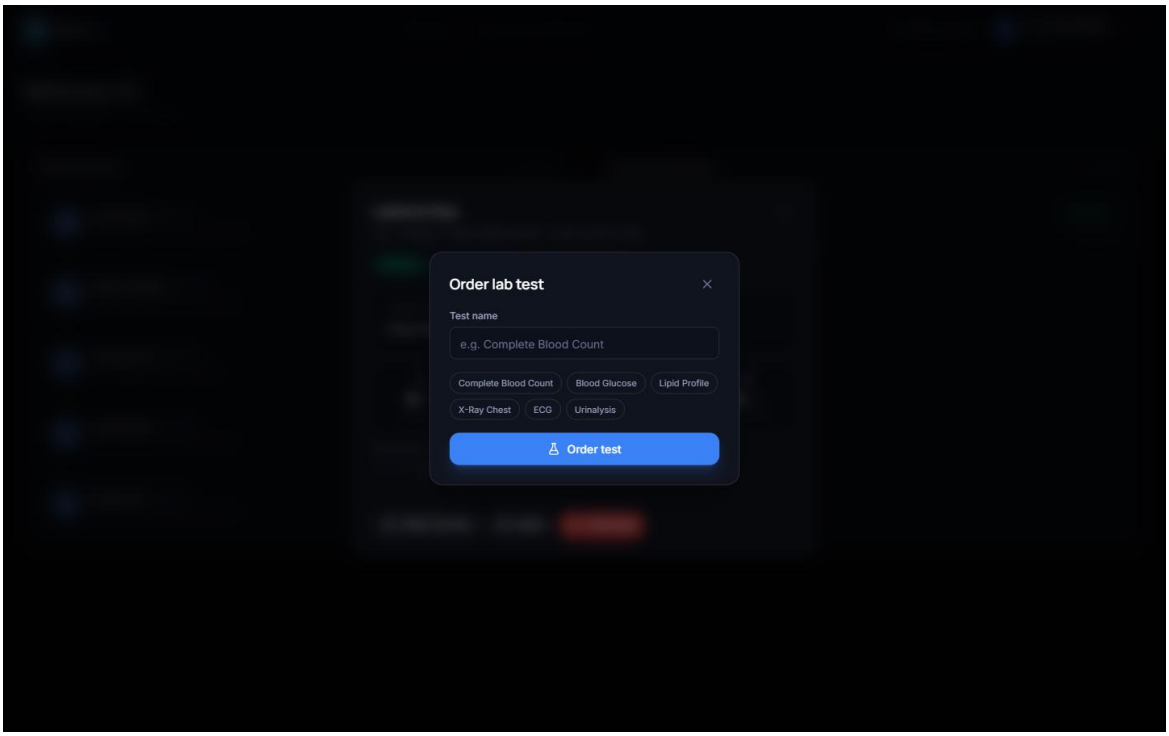


Figure 8.7 Laboratory order

A named test can be ordered from the consultation.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.8 Admission Selection

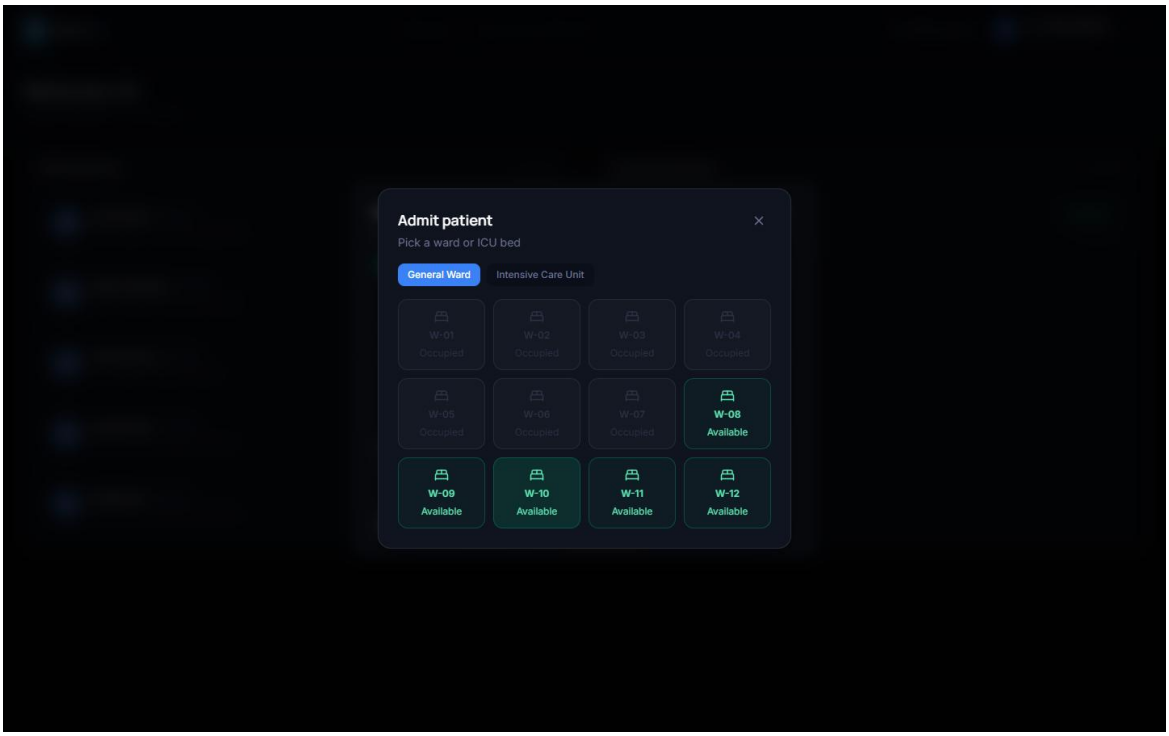


Figure 8.8 Admission modal

Availability is visible before selection.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.9 Ward Capacity

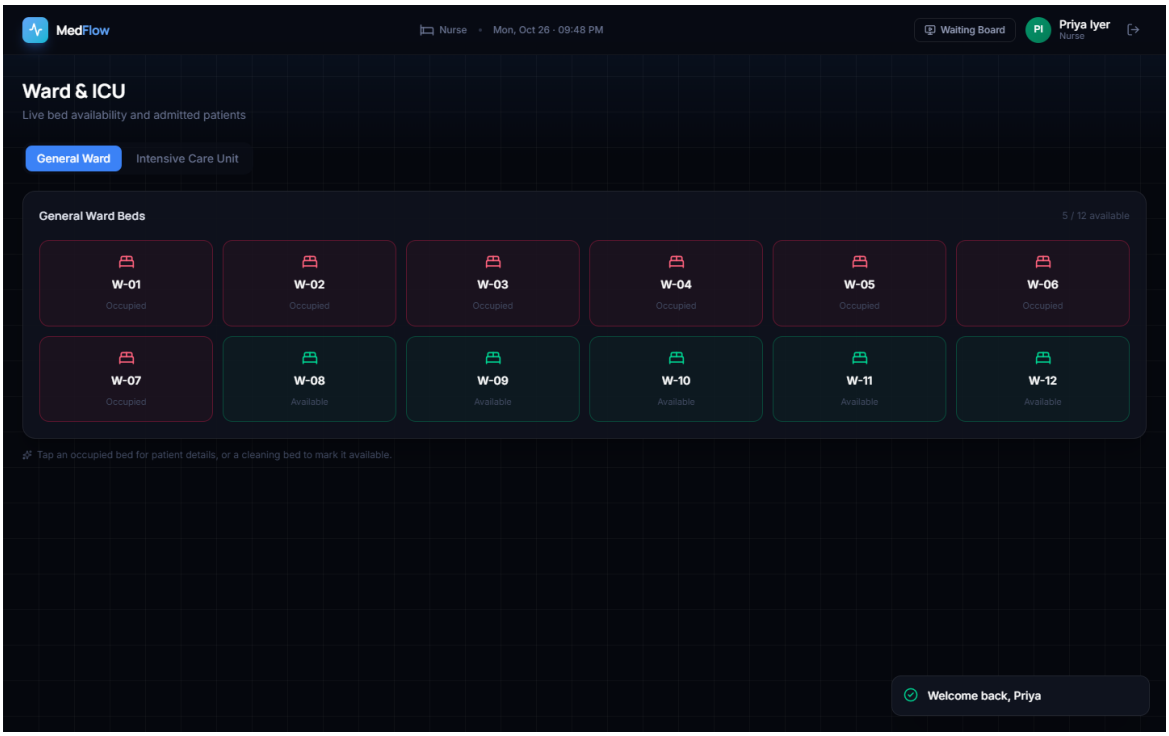


Figure 8.9 Ward bed board

Seven of twelve demonstration ward beds are occupied.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.10 ICU Capacity

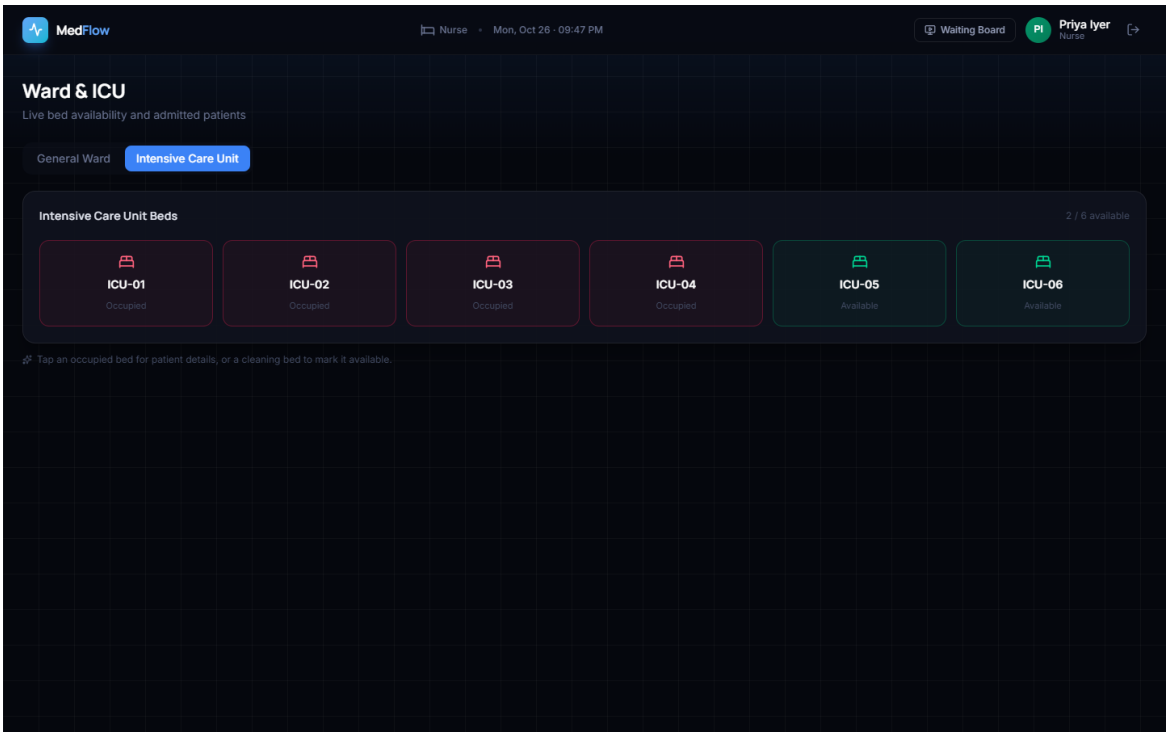


Figure 8.10 ICU bed board

Four of six demonstration ICU beds are occupied.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.11 Laboratory Workflow

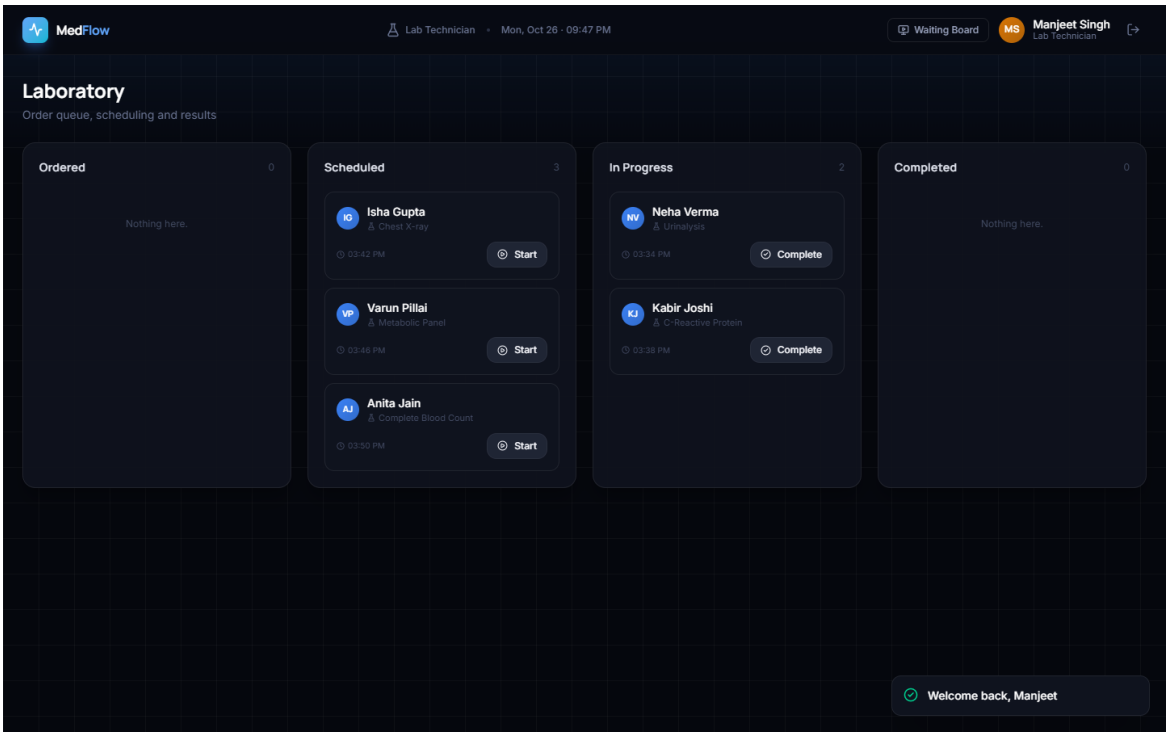


Figure 8.11 Laboratory dashboard

Five synthetic tests populate scheduled and in-progress work.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.12 Model Results

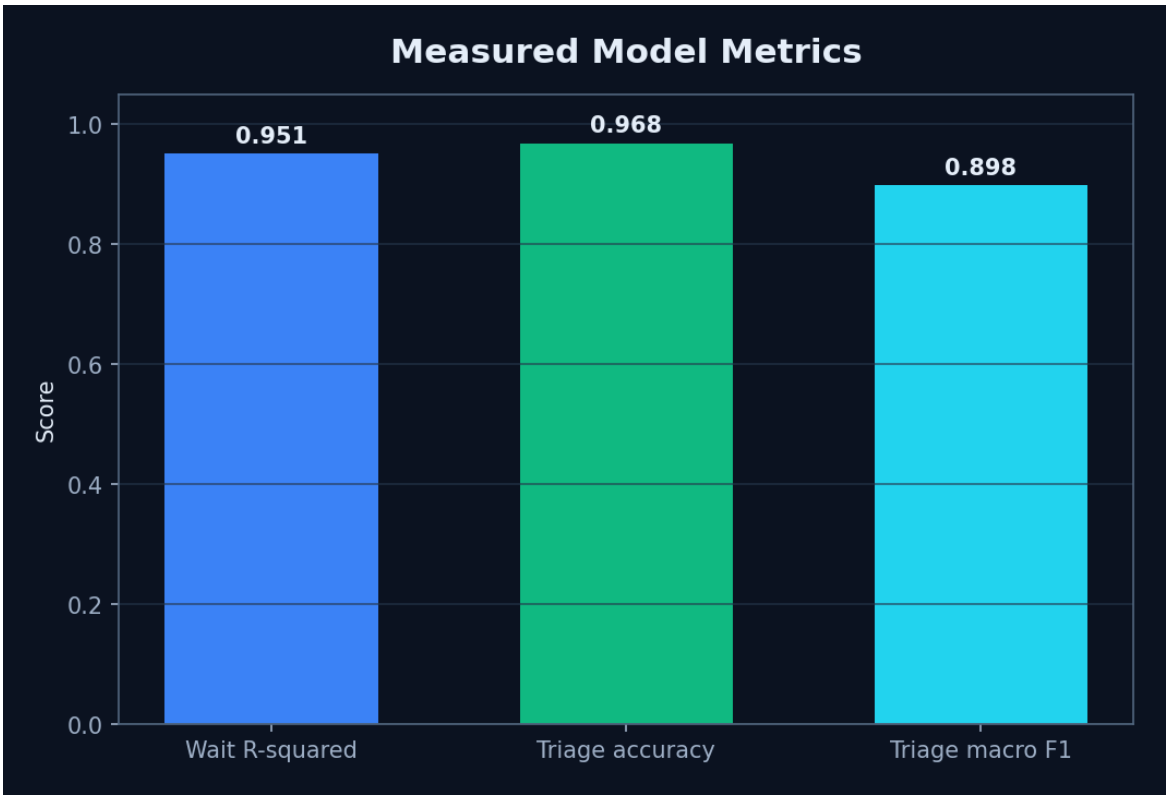


Figure 8.12 Model metrics

Scores summarise generator-matched test performance.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.13 Confusion and Failure Analysis

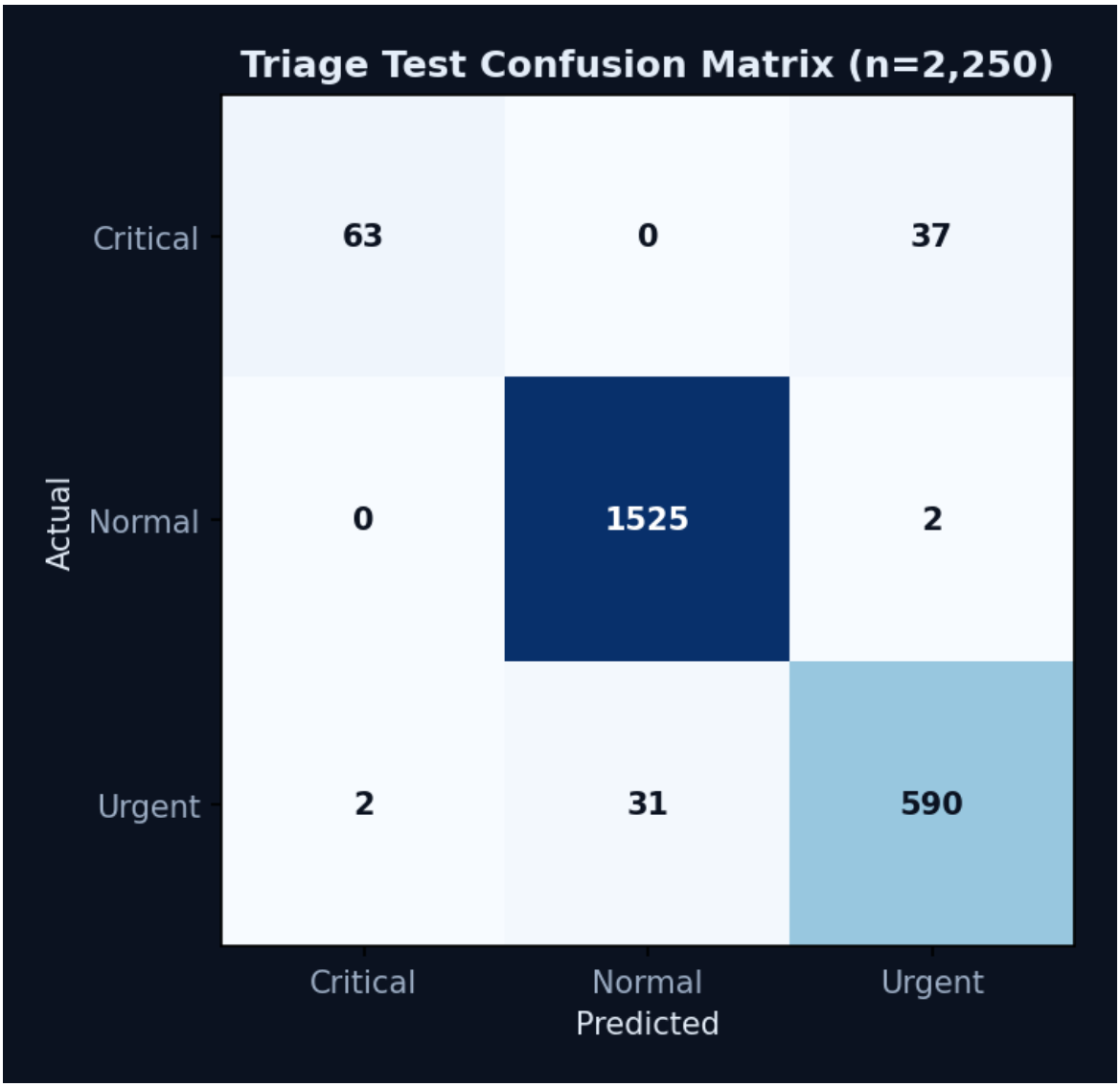


Figure 8.13 Triage confusion matrix

Critical recall of 63% is the main model failure finding.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.14 API and Capacity Results

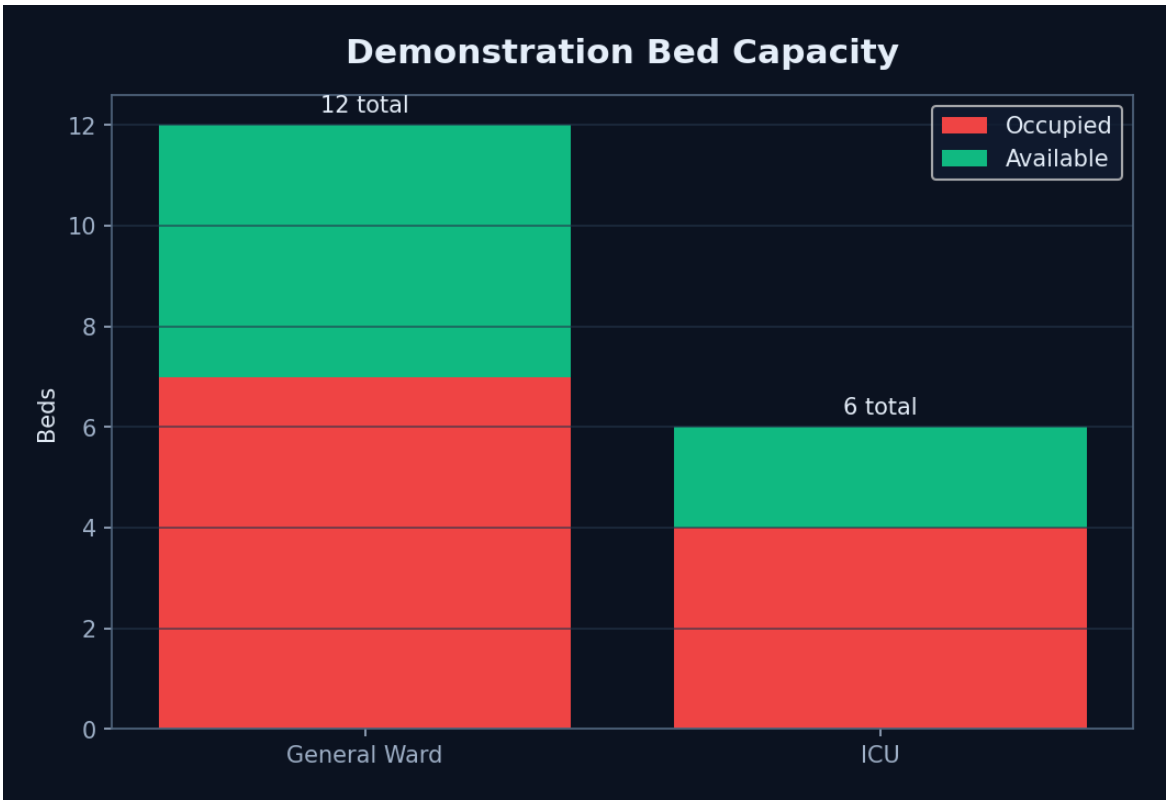


Figure 8.14 Demonstration bed capacity

Capacity is a synthetic application snapshot, not hospital utilisation evidence.

CHAPTER 8 RESULTS AND OUTPUT SCREENS

8.15 Objectives versus Outcomes

Table 8.1 Objectives versus outcomes.

Objective	Outcome
Tokens	Achieved
Movement	Partial
Lab workflow	Achieved; no optimisation evidence
Bed visibility	Achieved
AI assistance	Prototype achieved on synthetic data
Role dashboards	Achieved

Tokens, role dashboards, model integration, laboratory state and bed visibility were achieved as prototype functions. Movement tracking is partial, laboratory scheduling is not optimised and waiting-time reduction was not measured. The correct conclusion is functional integration with explicit gaps.

CHAPTER 9 CONCLUSION

9.1 Summary and Objectives Achieved

CareFlow implements one coherent patient-flow prototype across registration, queue, consultation, laboratory and bed state. The application executed successfully with populated data, all measured reads returned HTTP 200 and the frontend built successfully.

The model figures are reproducible but synthetic. The project's valid contribution is an integrated operational design and measured prototype, not a clinical or deployment claim.

CHAPTER 9 CONCLUSION

9.2 Findings, Applications and Limitations

Priority-aware sorting, shared visit state and live role views are practical foundations for a future hospital operations tool. The same architecture could support outpatient centres, diagnostic facilities or teaching simulations.

Critical recall, absent real data, missing tests, local-only persistence, security gaps and incomplete transition coupling define the present boundary.

CHAPTER 10 FUTURE SCOPE

10.1 Improvements and Further Research

Next work should obtain governed, de-identified operational data; use temporal and external validation; optimise critical recall; calibrate prediction uncertainty; model service capacity explicitly; and conduct clinician-led usability and safety assessment.

Automated unit, integration and end-to-end tests should cover every role and state transition.

CHAPTER 10 FUTURE SCOPE

10.2 Scalability, Deployment and Integration

A production redesign should use PostgreSQL, a managed message broker, authenticated multi-instance live channels, TLS, secret management, audit trails, backups and observability. HL7 FHIR interfaces could connect registration, laboratory, admission and EHR systems.

Demand forecasting and staffing recommendations should be introduced only after prospective evaluation with real operational outcomes.

REFERENCES

[1] S. A. N. Saritha, G. Kalnoor, P. R. Bhat, P. A. Rao, P. Prajwal, and P. T. Pradeep, 'Alleviate: An AI-Powered Mobile Application for Personalized Healthcare Management and Doctor Recommendation,' IEEE Access, vol. 13, pp. 215822–215832, 2025, doi: 10.1109/ACCESS.2025.3647089.

[2] S.-K. Tan, S.-C. Chong, K.-K. Wee, and L.-Y. Chong, 'Personalized healthcare: A comprehensive approach for symptom diagnosis and hospital recommendations using AI and location services,' Journal of Informatics and Web Engineering, vol. 3, no. 1, pp. 117–135, 2024.

[3] M. Abinaya and R. Ganesan, 'Effective search mechanism for finding nearest healthcare facilities,' in Proc. Global Conference on Communication Technologies, 2015, pp. 534–538.

[4] M. W. Munir, S. M. Omair, and M. Z. U. Haque, 'An Android based application for determine a specialized hospital nearest to patient's location,' International Journal of Computer Applications, vol. 118, no. 9, pp. 43–46, 2015.

[5] R. A. Nimbalkar and R. A. Fadnavis, 'Domain specific search of nearest hospital and healthcare management system,' in Proc. Recent Advances in Engineering and Computational Sciences, 2014, pp. 1–5.

[6] A. Kumar, 'Disease prediction and doctor recommendation system using machine learning approaches,' International Journal for Research in Applied Science and Engineering Technology, vol. 9, no. 7, pp. 34–44, 2021.

[7] L. Bjorkdahl, O. Pauli, J. Ostman, C. Ceccobello, S. Lundell, and M. Kjellberg, 'Towards holistic disease risk prediction using small language models,' arXiv:2408.06943, 2024.

REFERENCES

[8] R. Umbare, R. Patil, T. Mukund, A. Rathod, and P. Rajurkar, 'A smart healthcare application using artificial intelligence and machine learning,' in Proc. ICIPCN, 2024, pp. 204–210.

[9] D. Gujar, R. Biyani, T. Bramhane, S. Bhosale, and T. P. Vaidya, 'Disease prediction and doctor recommendation system,' International Research Journal of Engineering and Technology, vol. 5, pp. 3207–3209, 2018.

[10] D. Mogaveera, V. Mathur, and S. Waghela, 'E-health monitoring system with diet and fitness recommendation using machine learning,' in Proc. ICICT, 2021, pp. 694–700.

[11] P. Tanwar, K. Bansal, A. Sharma, and N. Dagar, 'AI based chatbot for healthcare using machine learning,' in Proc. ICAICCIT, 2023, pp. 751–755.

[12] R. Surve, T. Purohit, R. Joseph, and P. Shaikh, 'Healthcare chatbot using machine learning and NLP,' in Proc. ICAST, 2023, pp. 411–416.

[13] H. Mendapara, S. Digole, M. Thakur, and A. Dange, 'AI based healthcare chatbot system using natural language processing,' International Journal of Scientific Research and Engineering Development, vol. 4, no. 2, pp. 89–96, 2021.

[14] M. M. Rahman et al., 'Web-based multiple disease prediction system with transfer learning and healthcare chatbot,' in Proc. IEEE ICMI, 2024, pp. 1–5.

REFERENCES

[15] J. Tseng et al., 'An interactive healthcare system with personalized diet and exercise guideline recommendation,' in Proc. TAAI, 2015, pp. 525–532.

[16] T. Bhanuteja, K. V. N. Kumar, K. S. Poornachand, C. C. V. N. Ashish, and P. Anudeep, 'Symptoms based multiple disease prediction model using machine learning approach,' IJITEE, vol. 10, no. 9, pp. 67–72, 2021.

[17] 'Waiting Time Prediction in Queue Management: Leveraging Machine Learning Approach,' in Proc. International Conference on Recent Progresses in Science, Engineering and Technology, 2024.

[18] 'Prediction of outpatient waiting time using machine learning in a tertiary children's hospital,' Translational Pediatrics, 2023.

[19] 'Dissatisfaction-considered waiting time prediction for outpatients with interpretable machine learning,' Health Care Management Science, 2024.

APPENDIX A: IMPORTANT CODE

A.1 Queue ordering

The excerpt below is condensed from the checked-in implementation; ellipses indicate omitted arguments rather than new logic.

```
PRIORITY_RANK = {critical: 0, urgent: 1, normal: 2}
def sort_key(visit):
    return (PRIORITY_RANK.get(visit.priority, 3), visit.token_number)
```

APPENDIX A: IMPORTANT CODE

A.2 Wait prediction features

The excerpt below is condensed from the checked-in implementation; ellipses indicate omitted arguments rather than new logic.

```
X = DataFrame([{
  "department_type": dept_code, "queue_ahead": queue_ahead,
  "hour": hour, "priority": priority_code,
  "avg_service_minutes": avg_service_minutes, "servers": servers
}])
prediction = model.predict(X)[0]
```

APPENDIX A: IMPORTANT CODE

A.3 Visit creation orchestration

The excerpt below is condensed from the checked-in implementation; ellipses indicate omitted arguments rather than new logic.

```
priority_label, confidence = predict_triage(...)
ahead = queue_ahead_count(db, department.id, priority)
predicted_wait = predict_wait_minutes(...)
db.add(visit); db.add(MovementLog(...)); db.commit()
await manager.broadcast("visit_created", visit_out.model_dump())
```

APPENDIX A: IMPORTANT CODE

A.4 Role enforcement

The excerpt below is condensed from the checked-in implementation; ellipses indicate omitted arguments rather than new logic.

```
def require_roles(*roles):
    def checker(user = Depends(get_current_user)):
        if user.role not in roles:
            raise HTTPException(status_code=403)
        return user
    return checker
```

APPENDIX B: INSTALLATION AND USER GUIDE

B.1 Local Installation

Backend: enter 8.ProjectCode/backend, activate the supplied virtual environment or create one, install requirements.txt and ensure medflow.db contains the desired data. Frontend: enter 8.ProjectCode/frontend and run npm install if dependencies are absent.

The project stores models under backend/app/ml/artifacts. Running `python -m app.ml.train` regenerates them and changes the artefact files; do so only when intentional.

APPENDIX B: INSTALLATION AND USER GUIDE

B.2 Start and Sign In

Start the backend with `python -m uvicorn app.main:app --port 8010`. Start the frontend with `npm run dev`; open http://127.0.0.1:5173. Use the demo role account shown on the login screen. The public waiting board is /board.

The code contains no run.bat. Stop both development servers with Ctrl+C.

APPENDIX B: INSTALLATION AND USER GUIDE

B.3 Role Workflows

Reception creates a registration; doctors open a queue card and start consultation; doctors may order a test or choose an available bed; laboratory staff advance test work; nurses select a ward or ICU tab to inspect beds; administrators monitor KPIs.

Use only synthetic demonstration information unless appropriate privacy, consent and institutional controls are established.

APPENDIX C: ADDITIONAL SCREENS

C.1 Staff Sign-In

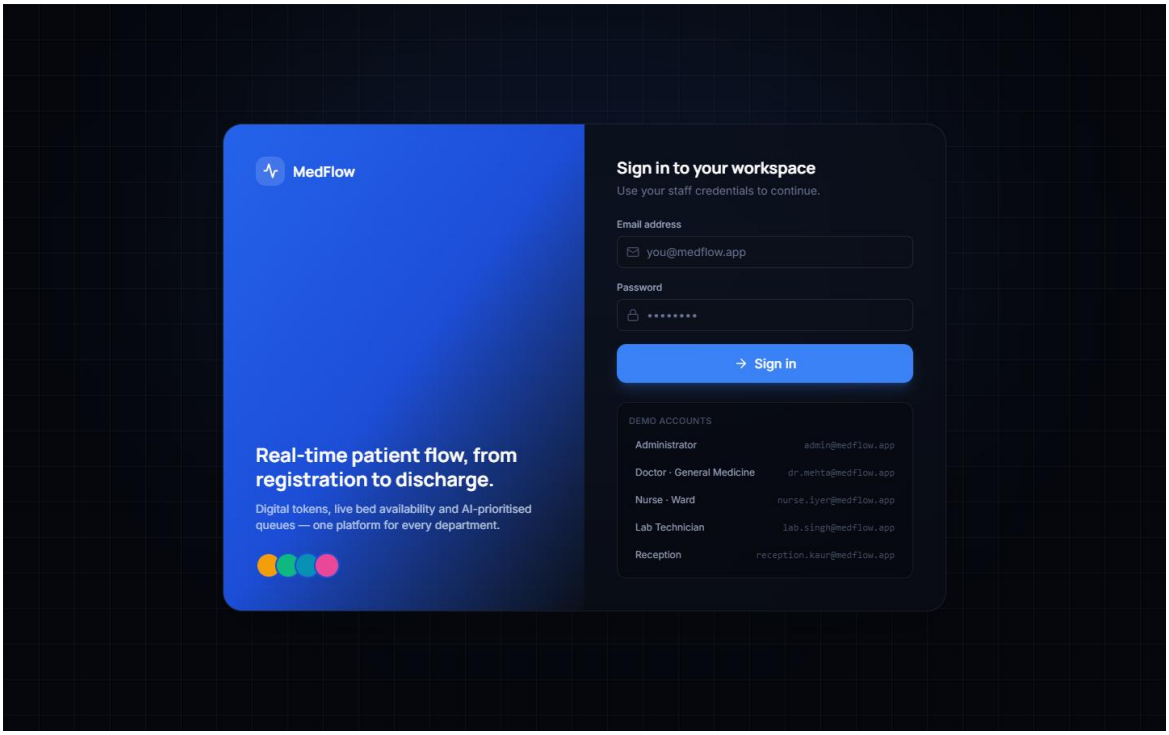


Figure C.1 Staff sign-in screen.

This is an actual capture of the running prototype using synthetic demonstration data.

APPENDIX C: ADDITIONAL SCREENS

C.2 Registration Detail

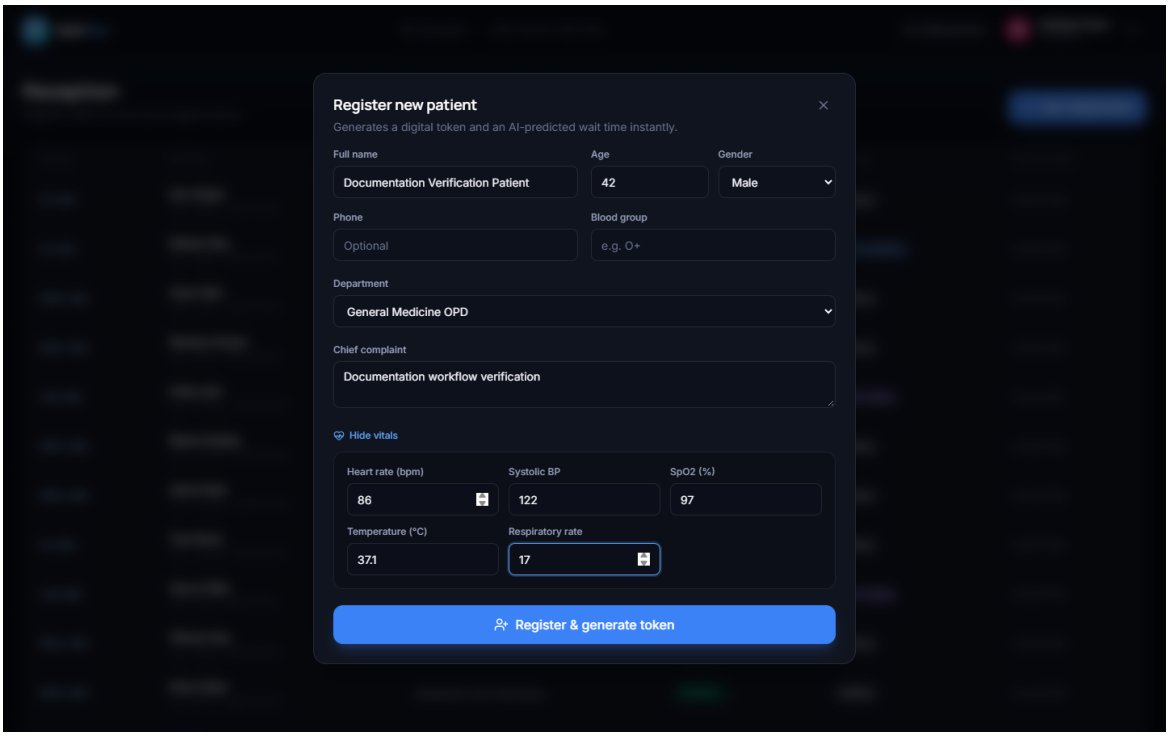


Figure C.2 Populated registration form.

This is an actual capture of the running prototype using synthetic demonstration data.

APPENDIX C: ADDITIONAL SCREENS

C.3 Patient Detail

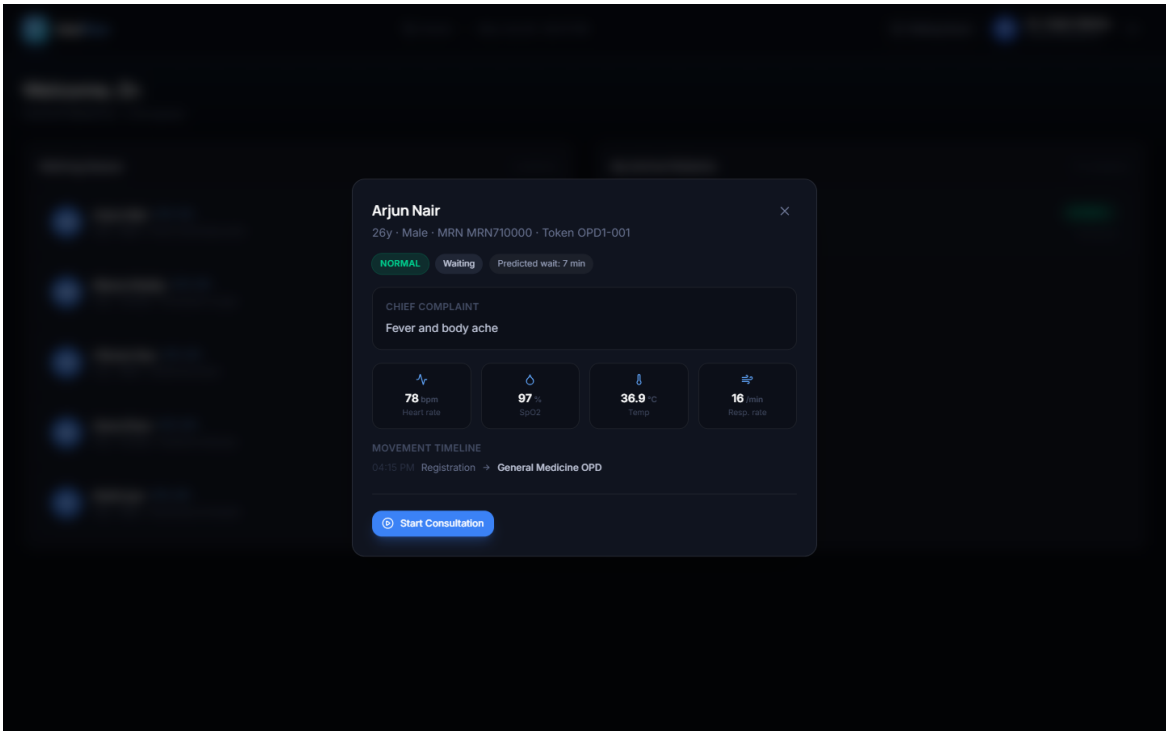


Figure C.3 Patient detail and movement.

This is an actual capture of the running prototype using synthetic demonstration data.

APPENDIX C: ADDITIONAL SCREENS

C.4 Admission Detail

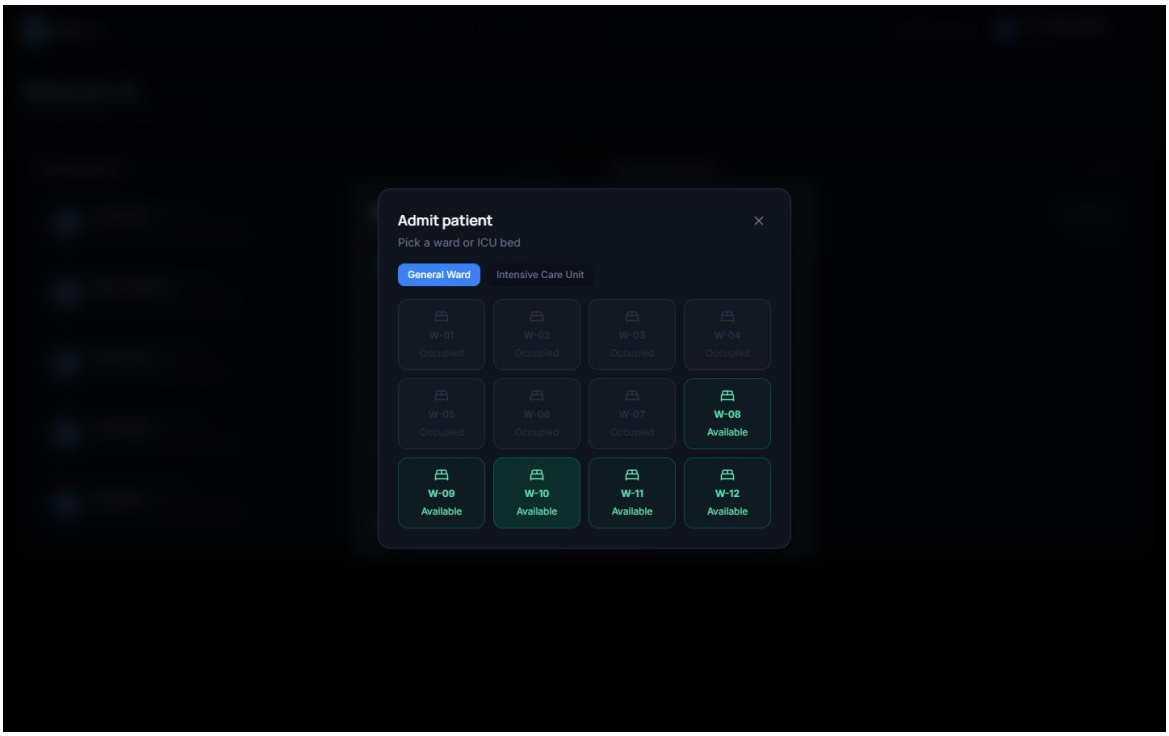


Figure C.4 Available-bed selection.

This is an actual capture of the running prototype using synthetic demonstration data.